

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ.....	8
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	10
ВВЕДЕНИЕ.....	13
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	16
1.1 Семантическая сеть как модель представления знаний.....	16
1.2 Модели представления графовых данных.....	17
1.3 Обзор существующих распределённых хранилищ.....	18
1.4 Сравнительный анализ и незаполненная ниша.....	20
1.5 Требования к разрабатываемой системе.....	21
1.5.1 Функциональные требования.....	21
1.5.2 Нефункциональные требования.....	22
2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ.....	23
2.1 Общая структура системы.....	23
2.2 Модель данных и хранение.....	24
2.3 Распределение данных между узлами.....	25
2.4 Репликация и консенсус.....	27
2.5 Распределённые транзакции.....	28
2.5.1 Гибридные логические часы и оракул точного времени.....	29
2.5.2 Сериализуемая изоляция снимков.....	30
2.5.3 Двухфазная фиксация по схеме Percolator.....	31
2.5.4 Внешняя согласованность через ожидание подтверждения.....	32
2.6 Обнаружение распределённых взаимоблокировок.....	33
2.7 Хранилище метаданных кластера.....	35
2.8 Безопасность взаимодействий.....	36
3 РЕАЛИЗАЦИЯ ПРОТОТИПА.....	37
3.1 Технологический стек.....	37
3.2 Модули плоскости данных.....	37
3.3 Модули управляющей плоскости.....	39
3.4 Сетевые протоколы.....	41
3.5 Развёртывание прототипа.....	41
3.6 Дисциплина разработки и стратегия тестирования.....	42
3.7 Наблюдаемость и конфигурация.....	45
4 ТЕСТИРОВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ.....	47
4.1 Тестовое окружение.....	47
4.2 Функциональное тестирование.....	47

4.3 Нагрузочное тестирование.....	48
4.3.1 Обходы графа.....	48
4.3.2 Векторный поиск.....	50
4.3.3 Поиск кратчайших путей.....	50
4.3.4 Задержка фиксации транзакций.....	51
4.3.5 Сквозная нагрузка.....	52
4.4 Тестирование отказоустойчивости.....	52
4.4.1 Сценарии тестирования отказов.....	52
4.4.2 Восстановление при отказе узла.....	53
4.4.3 Обнаружение взаимоблокировок.....	54
4.5 Сравнение стратегий размещения данных.....	55
4.6 Достоинства и недостатки разработанного решения.....	56
4.6.1 Достоинства.....	56
4.6.2 Недостатки и ограничения.....	57
ЗАКЛЮЧЕНИЕ.....	58
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	62
ПРИЛОЖЕНИЕ А Фрагменты определений программных интерфейсов.....	67
ПРИЛОЖЕНИЕ Б Разработанное программное обеспечение.....	70

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

- 2PC – двухфазная фиксация транзакции (two-phase commit)
- ACID – атомарность, согласованность, изоляция, устойчивость (atomicity, consistency, isolation, durability)
- BFS – обход графа в ширину (breadth-first search)
- CAP – теорема о согласованности, доступности и устойчивости к разделением сети (consistency, availability, partition tolerance)
- СМН – алгоритм Чанди–Мисры–Хааса обнаружения распределённых взаимоблокировок
- CRUD – создание, чтение, обновление, удаление (create, read, update, delete)
- DFS – обход графа в глубину (depth-first search)
- E2E-тестирование – сквозное тестирование (end-to-end testing)
- GPS – система глобального позиционирования (global positioning system)
- gRPC – удалённый вызов процедур Google (gRPC remote procedure calls)
- HLC – гибридные логические часы (hybrid logical clock)
- HNSW – иерархический навигируемый граф соседей (hierarchical navigable small world)
- HTTP – протокол передачи гипертекста (hypertext transfer protocol)
- HTTPS – защищённый протокол передачи гипертекста (hypertext transfer protocol secure)
- JWT – веб-токен JSON (JSON web token)
- kNN – k ближайших соседей (k nearest neighbors)
- KV – ключ-значение (key-value)
- LPG – граф с размеченными свойствами (labeled property graph)
- LSM – журналируемое слиянное дерево (log-structured merge tree)
- mTLS – взаимный TLS (mutual transport layer security)
- MVCC – многоверсионный контроль конкуренции (multi-version concurrency control)
- NSGA-II – генетический алгоритм недоминируемой сортировки второго поколения (non-dominated sorting genetic algorithm II)
- NTP – протокол сетевого времени (network time protocol)
- OWL – язык веб-онтологий (web ontology language)
- PQ – квантизация на основе произведения (product quantization)
- PTP – протокол точного времени (precision time protocol)

RDF – модель описания ресурсов (resource description framework)
RPC – удалённый вызов процедур (remote procedure call)
SCC – сильно связная компонента графа (strongly connected component)
SI – изоляция снимков (snapshot isolation)
SPARQL – язык запросов к RDF-данным (SPARQL protocol and RDF query language)
SQL – язык структурированных запросов (structured query language)
SSI – сериализуемая изоляция снимков (serializable snapshot isolation)
TLS – протокол безопасности транспортного уровня (transport layer security)
TSO – оракул точного времени (timestamp oracle)
БД – база данных
СУБД – система управления базами данных

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Семантическая сеть – модель представления знаний, в которой понятия описываются в виде узлов направленного графа, а смысловые отношения между понятиями в виде типизированных рёбер

Граф знаний – крупная семантическая сеть с явно описанной онтологией, объединяющая факты, понятия и отношения для решения прикладных задач поиска, рекомендаций и логического вывода

Граф с размеченными свойствами – модель данных, в которой каждой вершине и каждому ребру могут быть назначены тип и произвольный набор именованных свойств

Распределённая система управления базами данных – программный комплекс, выполняющий хранение и обработку данных на нескольких независимых вычислительных узлах, объединённых сетью, и предоставляющий клиенту единый логический интерфейс

Горизонтальное масштабирование – увеличение вычислительной ёмкости системы путём добавления новых узлов в кластер, в противоположность вертикальному масштабированию через наращивание ресурсов одного узла

Плоскость данных – слой распределённой системы, обрабатывающий горячий путь запросов: хранение данных, репликация, исполнение транзакций и графовых вычислений; горизонтально масштабируется добавлением узлов

Управляющая плоскость – слой распределённой системы, отвечающий за функции, не находящиеся на горячем пути запросов: маршрутизация и оптимизация запросов, размещение шардов, эволюция схемы; реализуется без сохранения состояния

Хранилище метаданных кластера – отдельная согласованная подсистема, хранящая сведения об устройстве кластера: членство узлов, карту шардов, аренды, версии схемы, реплицированный граф ожиданий транзакций

Шард – независимая часть распределённой базы данных, обслуживающая непрерывный диапазон ключей и реплицируемая независимо от других частей

Шардирование – стратегия распределения данных между независимыми узлами кластера с целью параллельной обработки и горизонтального масштабирования

Семантическое размещение данных – стратегия шардирования, при которой связанные по смыслу вершины графа размещаются на одном узле для снижения числа межузловых обращений при обходах

Сообщество в графе – плотно связанная подгруппа вершин, выделяемая методами обнаружения сообществ как кластер с высокой внутренней связностью и слабыми связями вовне

Реплика – копия данных на отдельном узле, обеспечивающая отказоустойчивость и параллельное обслуживание операций чтения

Консенсус – алгоритм, обеспечивающий согласование группой узлов единого порядка операций при наличии отказов узлов и потерь сообщений

Raft – алгоритм консенсуса с явным выделением лидера группы, описанный Д. Онгаро и Дж. Аустерхаутом, использующий последовательную репликацию журнала операций и применяемый в качестве промышленного стандарта для распределённой репликации

Свидетельствующий узел – узел консенсус-группы, участвующий в голосовании кворума, но не хранящий журнал и состояние реплики, что позволяет растягивать группу на удалённые регионы без значительных требований к пропускной способности канала связи

Транзакция – единица согласованного изменения базы данных, объединяющая несколько операций, которые выполняются как одно неделимое целое

Уровень изоляции транзакций – степень видимости промежуточных результатов одной транзакции для других транзакций, выполняющихся параллельно

Сериализуемость – наивысший уровень изоляции, при котором результат параллельного выполнения транзакций эквивалентен некоторому последовательному порядку их выполнения

Изоляция снимков – уровень изоляции, при котором каждая транзакция работает с согласованным снимком данных на момент её начала, без блокировок чтения

Внешняя согласованность – свойство распределённой системы, при котором порядок фиксации транзакций согласован с реальным временем их завершения

Линеаризуемость – свойство, при котором каждая операция выглядит выполненной мгновенно в некоторый момент времени между её началом и завершением

Многоверсионный контроль конкуренции – метод управления параллельным доступом, при котором изменения сохраняются как новые версии данных, а параллельные читатели работают с предыдущими согласованными версиями

Гибридные логические часы – механизм назначения временных меток, комбинирующий физические часы и счётчик-разделитель, обеспечивающий монотонность и согласованность с причинно-следственным порядком событий

Оракул точного времени – служба, возвращающая интервал неопределённости физического времени, в пределах которого гарантированно

лежит истинное системное время; используется как основа для обеспечения внешней согласованности через ожидание подтверждения

Взаимоблокировка – циклическая зависимость между транзакциями, при которой каждая ждёт освобождения ресурса, удерживаемого другой транзакцией из цикла

Граф ожиданий – направленный граф, вершинами которого являются активные транзакции, а рёбрами — отношения ожидания одной транзакции освобождения ресурса другой

Сильно связная компонента – максимальное подмножество вершин направленного графа, в котором из каждой вершины существует путь до любой другой вершины подмножества

ВВЕДЕНИЕ

В современных информационных системах данные нередко имеют сложную сетевую структуру: связи между сущностями значимы не меньше, чем сами сущности. Для представления и хранения таких данных применяются графовые модели — направленные графы, в которых вершинам соответствуют сущности, а рёбрам — типизированные связи между ними. Когда такая структура описывает фактическое и понятийное знание о некоторой предметной области, говорят о семантической сети, а её крупная реализация с явной онтологией о графе знаний.

Графы знаний применяются во многих прикладных областях:

- рекомендательные системы: построение рекомендаций по графу взаимодействий пользователей и сущностей;
- поиск семантически связанной информации;
- логический вывод по онтологиям в системах автоматизированного принятия решений;
- биомедицинские исследования и анализ молекулярных взаимодействий;
- анализ социальных графов.

Современные открытые и корпоративные графы знаний достигли объёмов в десятки миллиардов фактов. При таких объёмах хранение и обработка графа на одном вычислительном узле невозможны как по причинам ёмкости накопителей, так и по причинам пропускной способности. Возникает потребность в распределённой системе хранения, способной разместить граф на множестве узлов кластера и параллельно обслуживать запросы.

К такой системе предъявляется ряд требований:

- 1) графовая модель данных с типизированными вершинами и рёбрами и произвольным набором именованных свойств;
- 2) горизонтальное масштабирование — увеличение ёмкости и пропускной способности через добавление новых узлов в кластер;
- 3) сериализуемая изоляция многошардовых транзакций;
- 4) внешняя согласованность операций фиксации — согласие порядка фиксации транзакций с реальным временем их завершения.

Существующие распределённые системы управления данными покрывают эти требования лишь частично: графовые системы, как правило, ограничены в гарантиях распределённых транзакций; реляционные NewSQL-системы обеспечивают сильные транзакционные гарантии, но не поддерживают графовых операций на уровне хранения; триплетные хранилища RDF в распределённом режиме редко достигают сериализуемости.

Целью данной работы является обеспечение возможности горизонтального масштабирования хранилища данных семантической сети, поддерживающего сериализуемую изоляцию транзакций и внешнюю согласованность операций фиксации. Для достижения этой цели поставлены следующие задачи:

- 1) проведение анализа существующих подходов к распределённому хранению семантических графовых данных;
- 2) формулировка функциональных и нефункциональных требований к разрабатываемой системе;
- 3) проектирование архитектуры распределённого графового хранилища с обоснованием выбора алгоритмов и технологий;
- 4) реализация прототипа системы в соответствии с проектом;

- 5) разработка методики и реализация комплексного автоматизированного тестирования прототипа;
- 6) проведение нагрузочного тестирования и проверка отказоустойчивости прототипа;
- 7) экспериментальная оценка эффективности предложенного семантического размещения данных в сравнении с базовыми подходами.

Достижение поставленных задач позволит создать распределённое графовое хранилище, применимое в прикладных системах работы с графами знаний, подсистемах семантического поиска и логического вывода по онтологиям.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Семантическая сеть как модель представления знаний

Понятие семантической сети введено в работах М. Куилиана в конце 1960-х годов как структура для хранения и обработки концептуальной памяти [1]. В исходной форме семантическая сеть — это направленный граф, в котором вершины соответствуют концептам, а рёбра — типизированным отношениям. Эта модель оказалась удобной для представления знаний в системах искусственного интеллекта, поскольку явно фиксирует связи между фактами; на её основе развились методы фреймового представления знаний, концептуальной зависимости и формальные онтологические системы [2; 3; 4].

В современной интерпретации семантическая сеть используется в двух основных значениях. Первое — академическое — сохраняет исходное понимание сети как графа концептов с типизированными отношениями. Второе связано с инициативой Semantic Web консорциума W3C: набор стандартов RDF, SPARQL и OWL для описания, запроса и логического вывода по ресурсам сети Интернет. В настоящей работе термин используется в широком смысле как обобщённое название графовой модели представления знаний, объединяющей оба указанных значения. Объект, который должна поддерживать разрабатываемая система, направленный граф, вершины и рёбра которого несут типы и произвольный набор именованных свойств с возможностью многократных версий каждого факта во времени. Концептуальный пример показан на рисунке 1.

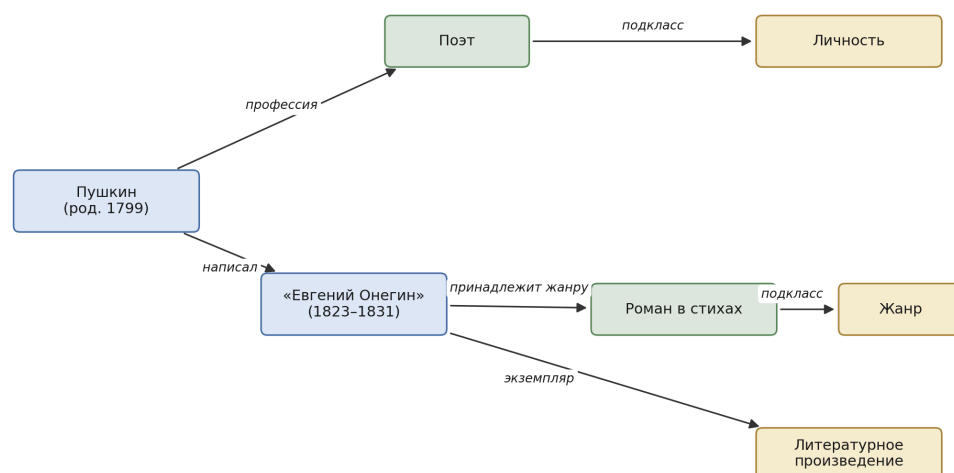


Рисунок 1 – Фрагмент семантической сети как направленного графа концептов с типизированными отношениями

Современные открытые графы знаний достигли значительных объёмов: RDF-представление Wikidata в сервисе запросов содержит порядка шестнадцати миллиардов триплетов, ежемесячные релизы DBpedia — свыше двадцати миллиардов триплетов [2; 5]; корпоративные графы знаний достигают сопоставимых объёмов [6]. Такие объёмы превышают возможности одного вычислительного узла как по дисковому пространству, так и по пропускной способности, поэтому обработка графа становится распределённой задачей. Современные сценарии работы с графами знаний всё чаще объединяют структурный обход графа с поиском по семантическим векторным представлениям вершин, что определяет дополнительное требование к хранилищу — нативная поддержка векторных свойств и поиска ближайших соседей наряду с графовыми обходами.

1.2 Модели представления графовых данных

Для хранения графовых данных в распределённых системах сложились две основные модели. Триплетная модель (Resource Description Framework, RDF) рассматривает граф как множество утверждений вида «субъект–предикат–объект»; восходит к стандартам W3C, использует SPARQL для запросов и OWL для логического вывода. Достоинствами

являются открытые стандарты и формальная семантика; недостатками неэффективное хранение свойств вершин (каждое свойство — отдельный триплет с вершиной-литералом) и слабая типизация рёбер. Модель размеченного графа свойств (Labeled Property Graph, LPG) присваивает произвольный набор именованных свойств каждой вершине и каждому ребру; применяется в большинстве промышленных графовых СУБД, обеспечивает более эффективное хранение и более выразительный язык запросов, но исторически развивалась вне стандартов W3C.

Модель LPG является надмножеством RDF: любой триплет «субъект–предикат–объект» представим в LPG как ребро от вершины-субъекта к вершине-объекту с типом, равным предикату [2; 4]. Обратное в общем случае неверно: представление LPG-графа в триплетном виде требует материализации рёбер как вершин-носителей свойств, что существенно увеличивает объём данных. Поэтому выбор LPG в качестве базовой модели обеспечивает поддержку обеих интерпретаций семантической сети — триплеты обрабатываются как частный случай рёбер. Сопоставление моделей показано на рисунке 2.

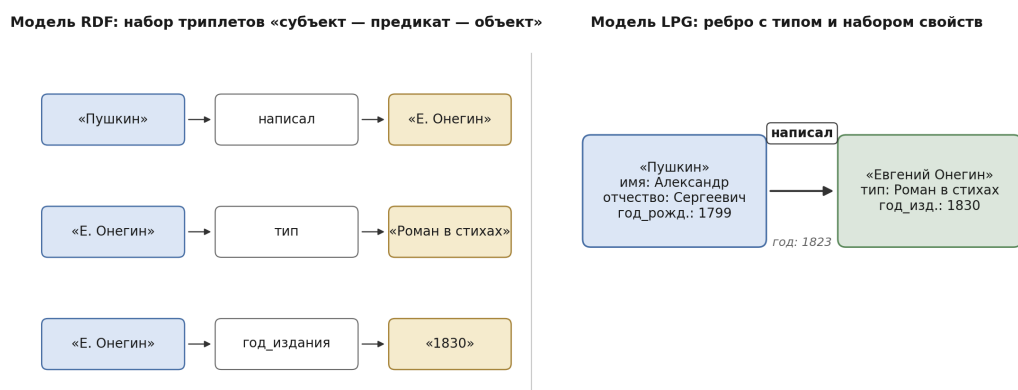


Рисунок 2 – Сопоставление моделей данных RDF и LPG

1.3 Обзор существующих распределённых хранилищ

Существующие распределённые системы, применимые для хранения графов знаний, разделяются на три архитектурных класса: триплетные

хранилища RDF, графовые системы с моделью LPG и реляционные распределённые системы NewSQL.

К первому классу относятся Virtuoso, Stardog и Blazegraph — все поддерживают SPARQL и OWL. Virtuoso Cluster Edition обеспечивает распределённое исполнение, но сериализуемая изоляция SPARQL-обновлений в нём не гарантируется. Stardog Cluster предоставляет ACID-транзакции с изоляцией снимков; распределённая сериализуемость не заявлена. Blazegraph в режиме репликации высокой доступности теряет сериализуемые гарантии (типичный AP-выбор в смысле теоремы CAP). Общей особенностью класса являются слабые гарантии распределённых транзакций и отсутствие внешней согласованности.

Ко второму классу относятся Neo4j, JanusGraph и Dgraph. Neo4j Enterprise по умолчанию обеспечивает READ COMMITTED; сериализуемость достигается через явные блокировки приложения. Neo4j Causal Cluster централизует запись на одном лидере, а механизм Fabric основан на федеративной маршрутизации, а не на нативном шардировании. JanusGraph реализует LPG поверх Cassandra или HBase, наследуя их транзакционные гарантии (редко достигают сериализуемости). Dgraph использует собственный Raft и заявляет сериализуемую изоляцию; шардирование выполняется по группам предикатов. Кроме того, к данному классу относятся исследовательские распределённые графовые системы G-Tran [7] и GraphScope Flex [8]. Экспериментальное сравнение производительности графовых СУБД этого класса приведено в работе [9]. Внешней согласованности ни одна из систем класса не предоставляет.

К третьему классу относятся Google Spanner и CockroachDB — не графовые, но обеспечивающие сильные транзакционные гарантии и нативное горизонтальное масштабирование, что делает их естественной точкой отсчёта [10; 11; 12]. Spanner обеспечивает строгую сериализуемость и внешнюю согласованность через аппаратно-программный механизм TrueTime

(GPS-приёмники и атомные часы). CockroachDB обеспечивает сериализуемость, но внешнюю согласованность не гарантирует, так как используются гибридные логические часы без аппаратного источника точного времени. Объявленная в 2024 году надстройка Spanner Graph предоставляет графовый язык запросов поверх реляционного хранилища, но модель хранения остаётся реляционной; в настоящем сравнении она не образует отдельной категории.

1.4 Сравнительный анализ и незаполненная ниша

Сравнение рассмотренных систем по ключевым признакам — модели данных, поддержке горизонтального распределения, уровню изоляции транзакций, наличию внешней согласованности и поддержке графовых операций на уровне хранения — приведено в таблице 1.

Таблица 1 – Сравнение существующих распределённых систем управления графовыми и реляционными данными

Система	Модель	Распределённость	Сериализуемость	Внешняя согласованность	Графовые операции
Virtuoso	RDF (триплеты)	частичная	частичная	нет	да
Stardog	RDF (триплеты)	да	частичная	нет	да
Blazegraph	RDF (триплеты)	частичная	частичная	нет	да
Neo4j Enterprise	LPG	частичная	частичная	нет	да
JanusGraph	LPG	да	нет	нет	да
Dgraph	LPG	да	да	нет	да
Spanner	реляционная	да	да	да	нет
CockroachDB	реляционная	да	да	частичная	нет

Ни одна из рассмотренных систем не удовлетворяет одновременно четырём ключевым требованиям. По компромиссу теоремы CAP к стороне доступности тяготеют Cassandra-бэкенд JanusGraph и ряд RDF-хранилищ; к стороне согласованности — Spanner, CockroachDB, Dgraph. Разрабатываемая система относится к стороне согласованности, при этом одновременно поддерживает графовую модель данных и внешнюю согласованность, то есть

сочетание, не обеспеченное ни одной из рассмотренных систем. Незаполненная функциональная ниша состоит в распределённом графовом хранилище с моделью LPG, которое одновременно обеспечивает сериализуемую изоляцию, внешнюю согласованность, нативное горизонтальное шардирование и поиск по семантическим векторным представлениям наряду с графовыми обходами. Закрытие этой ниши определяет направление настоящей работы.

1.5 Требования к разрабатываемой системе

Выявленный пробел определяет требования к системе.

1.5.1 Функциональные требования

- 1) модель размеченного графа свойств с типизированными вершинами и рёбрами, произвольным набором именованных свойств и хранением семантических векторных представлений для поиска ближайших соседей;
- 2) полный набор графовых операций: CRUD над вершинами и рёбрами с атомарностью каждой операции; обходы графа в ширину (BFS) и в глубину (DFS) с фильтрацией; поиск кратчайших путей; поиск k ближайших соседей (kNN); агрегация результатов и курсорная пагинация со стабильным срезом;
- 3) многоверсионное хранение с чтением исторических срезов в пределах настраиваемого периода удержания;
- 4) нативное горизонтальное распределение данных с автоматическим разделением и слиянием диапазонов под изменение нагрузки и репликацией каждого шарда с автоматическим выбором лидера;
- 5) атомарные многошардовые транзакции с тремя уровнями изоляции — READ_COMMITTED, изоляция снимков и сериализуемая изоляция (по умолчанию) — и внешней согласованностью фиксации;

- 6) автоматическое обнаружение распределённых взаимоблокировок и откат одной из транзакций цикла;
- 7) обязательное взаимное TLS-шифрование внутренних каналов связи и аутентификация клиентов через веб-токены.

1.5.2 Нефункциональные требования

- 1) производительность: операции точечного чтения и репликация записи внутри одного центра обработки данных должны выполняться с задержкой порядка единиц миллисекунд; задержка сквозной фиксации сериализуемой транзакции порядка десятков миллисекунд по 99-му перцентилю с учётом ожидания подтверждения при включённом оракуле точного времени;
- 2) масштабируемость: рост пропускной способности должен быть близок к линейному при добавлении узлов до целевого размера кластера;
- 3) отказоустойчивость: кластер должен продолжать обслуживать операции при потере не более половины узлов любой консенсус-группы; восстановление лидера консенсус-группы должно занимать менее пяти секунд;
- 4) целостность данных: гарантия отсутствия потерянных подтверждённых транзакций при отказах отдельных узлов;
- 5) наблюдаемость: экспорт метрик в формате Prometheus и распределённая трассировка запросов;
- 6) тестируемость: автоматическое покрытие модульными и интеграционными тестами не менее семидесяти процентов;
- 7) развёртываемость: автоматизированное развёртывание полного кластера одной командой.

2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ

Сформулированные требования определяют структуру архитектурного решения. Ниже описаны общая структура системы, модель данных и физическое хранение, схема распределения данных между узлами, репликация и консенсус, подсистема распределённых транзакций с механизмом обнаружения взаимоблокировок, а также архитектура хранилища метаданных кластера и решения по безопасности взаимодействий.

2.1 Общая структура системы

Система спроектирована по принципу разделения ответственности между тремя независимо масштабируемыми плоскостями: плоскостью данных, управляющей плоскостью и хранилищем метаданных кластера. Плоскость данных обрабатывает горячий путь запросов (хранение, репликация, транзакции, обходы графа) и горизонтально масштабируется добавлением узлов; каждый узел — единый исполняемый файл с уровнями хранения, консенсуса, менеджером транзакций и подсистемой графовых вычислений. Управляющая плоскость без сохранения состояния отвечает за функции вне горячего пути — маршрутизацию запросов через шлюз и автономные циклы оркестратора (размещение шардов, эволюция схемы, обнаружение длинных циклов ожидания). Хранилище метаданных кластера — отдельная Raft-группа из трёх голосующих узлов, размещённая внутри сервиса узла данных; такое совмещение позволяет использовать единую реализацию консенсуса и сократить число эксплуатируемых типов узлов до одного. Общая структура системы представлена на рисунке 3.

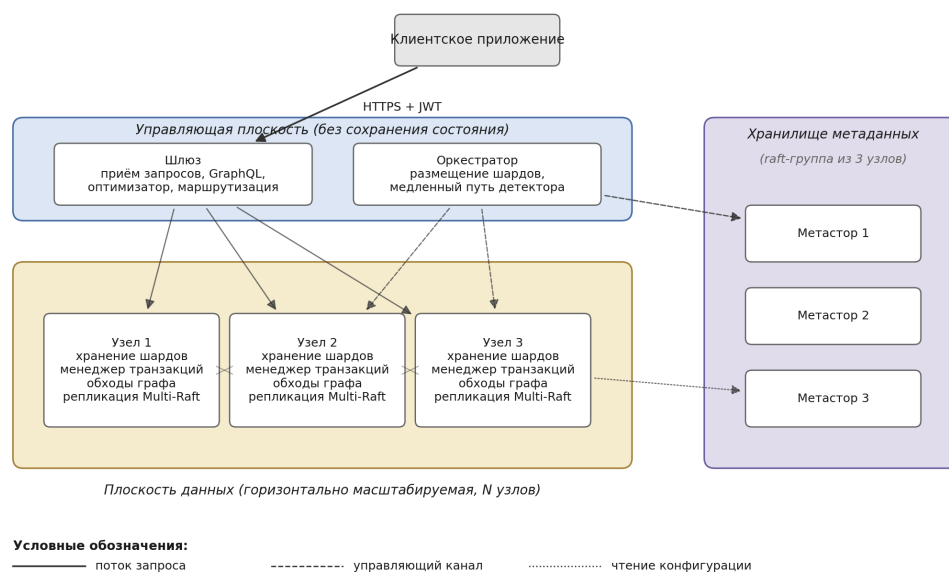


Рисунок 3 – Общая трёхплоскостная архитектура распределённого хранилища

Сопоставление плоскостей по основным характеристикам приведено в таблице 2.

Таблица 2 – Характеристики плоскостей системы

Плоскость	Язык реализации	Сохранение состояния	Расположение в запросе	Масштабирование
Плоскость данных	Rust	да	горячий путь	по числу узлов
Управляющая плоскость	Kotlin	нет	через шлюз	от 2 реплик
Хранилище метаданных	Rust	да	контрольный путь	фиксированный кворум из 3 голосующих узлов

2.2 Модель данных и хранение

В качестве модели данных выбран размеченный граф свойств: вершина характеризуется уникальным идентификатором, типом и произвольным набором именованных свойств, включая векторные представления; ребро описывается типом, парой инцидентных вершин, направлением и набором свойств.

Физическое хранение организовано на основе журналируемого слиянного дерева (LSM-tree); в качестве реализации используется библиотека RocksDB [13] как зрелое решение с механизмами компакции, восстановления и фильтрами Блума. Графовая семантика реализуется уровнем кодирования ключей над общим интерфейсом ключ-значение.

Данные разделены между несколькими семействами столбцов RocksDB, что позволяет независимо настраивать параметры компакции и кэширования. Вершины хранятся одним семейством по идентификатору; рёбра — двумя зеркальными семействами по исходящей и входящей вершине для симметричного обхода в обе стороны; исторические версии рёбер — двумя семействами многоверсионных рёбер с привязкой к временным меткам фиксации. Отдельные семейства выделены под векторные представления вершин для индекса HNSW, кэш идентификаторов идемпотентности и метаданные кластера, изолированные префиксом ключа от графовых данных.

Многоверсионность реализуется как часть схемы кодирования ключей [14; 15]: каждое изменение создаёт новую запись с временной меткой фиксации, а предыдущие версии сохраняются в течение настраиваемого периода удержания; фоновый процесс удаляет устаревшие версии. Для поиска по векторным представлениям реализован индекс HNSW [16]; горячая часть размещается в памяти, а для холодной применяется квантизация по произведению (PQ).

2.3 Распределение данных между узлами

Распределение данных организовано в два уровня: шардирование по диапазонам идентификаторов вершин и семантическое перераспределение границ диапазонов на основе обнаружения сообществ в графе обращений.

Шард определяется как непрерывный диапазон идентификаторов вершин вместе со всеми инцидентными рёбрами и векторными представлениями. Отображение «идентификатор → шард» вычисляется по границам диапазонов; обратное отображение «шард → узел-лидер» хранится

в хранилище метаданных и кэшируется шлюзами с ограниченным временем жизни кэша. Диапазонное шардирование выбрано, а не хэш-шардирование, поскольку графовые обходы — основная операция системы: при размещении соседних по идентификатору вершин в одном диапазоне большинство шагов обхода остаётся локальными, тогда как хэш-шардирование распределяет соседей вершины случайно между узлами и вызывает сетевое обращение почти на каждом шаге. Оркестратор автоматически разделяет шарды при росте нагрузки и сливает их при снижении.

Диапазонное шардирование само по себе не учитывает семантическую структуру графа: часто запрашиваемые вместе вершины могут попасть в разные диапазоны. Для устранения этого ограничения введён второй уровень распределения: оркестратор непрерывно подстраивает границы диапазонов так, чтобы сообщества графа обращений клиентов оказывались внутри одного диапазона. Обнаружение сообществ выполняется алгоритмом Лейдена (Leiden) [17; 18] — развитием метода Лувена, устраняющим проблему слабо связанных сообществ. Целевой функцией служит модулярность графа обращений с весами рёбер, пропорциональными частоте совместного обращения. Назначение сообществ на конкретные шарды и принятие решения о переносе вершин выполняется многокритериальной оптимизацией Парето по алгоритму NSGA-II [19; 20; 21] с тремя целями: минимизация доли межузловых обращений, балансировка нагрузки и минимизация стоимости миграции. Перенос выполняется только при положительной чистой выгоде. Миграция данных проходит онлайн через механизм переходного консенсуса Raft, описанный в §2.4. Двухуровневая схема распределения показана на рисунке 4.

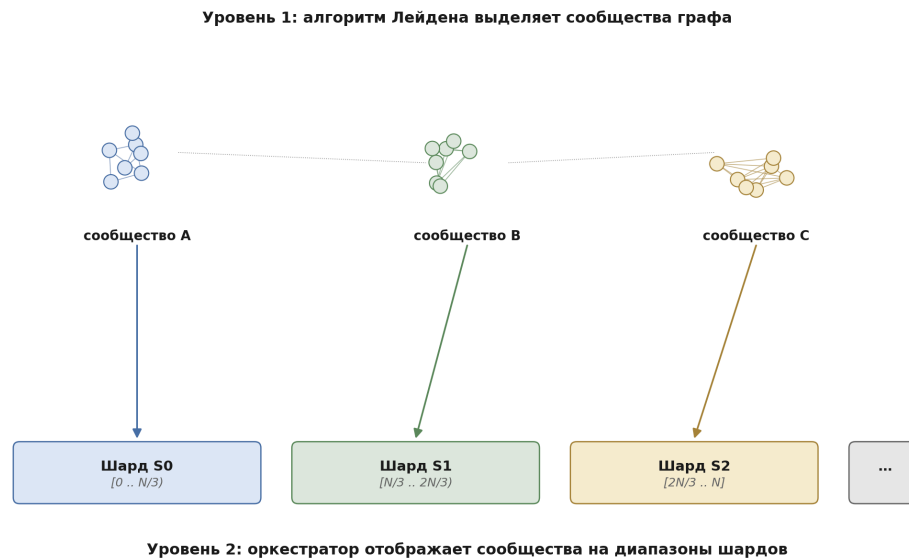


Рисунок 4 – Двухуровневое распределение данных по диапазонам идентификаторов и сообществам алгоритма Лейдена

2.4 Репликация и консенсус

Каждый шард реплицируется на три голосующих узла по умолчанию; согласованность реплик обеспечивается протоколом Raft [22; 23; 24; 25] через библиотеку *raft-rs*, использованную в проектах TiKV и etcd. Каждой консенсус-группе соответствует один шард данных — схема «распределённый Raft» (Multi-Raft). Один узел может одновременно участвовать в десятках групп в разных ролях (лидер, последователь, наблюдатель). Все группы исполняются на едином асинхронном исполнителе без выделения отдельного потока на каждую группу. Для оптимизации чтений применяются чтения по аренде: лидер обслуживает чтения локально в пределах продлеваемой аренды, длительность которой ограничена оценкой неопределённости системного времени от оракула точного времени (§2.5.1), что сохраняет линейризуемость без дополнительного обмена сообщениями. Расширение предголосования (Pre-Vote) защищает стабильную конфигурацию от срыва при кратковременных сетевых разделениях.

Изменение состава консенсус-группы — добавление узла, удаление или передача лидерства — выполняется через переходный консенсус: на время

изменения кворум определяется и по старому, и по новому составу одновременно, что исключает потерю доступности. Миграция шарда между узлами выполняется так: на узел-приёмник добавляется новая реплика, она загружает снимок состояния и догоняет журнал, после чего выполняется передача лидерства, а узел-источник удаляется из состава группы; шлюз обновляет кэш карты шардов по истечении TTL кэшированной записи. Архитектура одной консенсус-группы показана на рисунке 5.

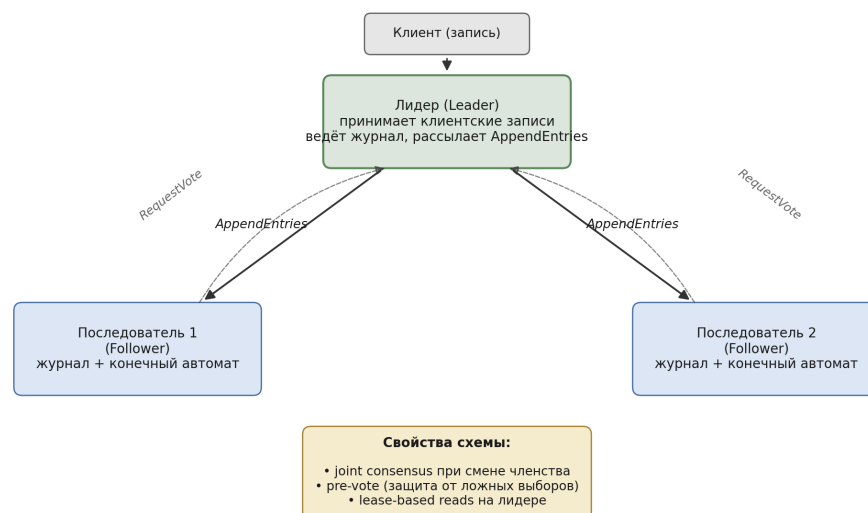


Рисунок 5 – Архитектура одной консенсус-группы Raft

Для географически распределённых развёртываний архитектурно предусмотрены свидетельствующие узлы, участвующие в голосовании кворума, но не хранящие журнал изменений; такая конфигурация позволяет растянуть группу на несколько регионов без существенных требований к пропускной способности межрегиональных каналов. Автоматическое управление региональным размещением реплик в текущей версии не реализовано и отнесено к перспективам развития системы.

2.5 Распределённые транзакции

Транзакционная подсистема обеспечивает три уровня изоляции по выбору клиента: READ_COMMITTED, изоляция снимков и сериализуемая изоляция (по умолчанию, с внешней согласованностью фиксации). При

READ_COMMITTED каждое чтение возвращает зафиксированное значение на момент исполнения; при изоляции снимков все чтения работают со снимком на момент *start_ts* без гарантии сериализуемости; при сериализуемой изоляции корректность обеспечивается комбинацией SSI и ожидания подтверждения над оракулом точного времени. Уровень изоляции хранится в реестре транзакций на узле создания; последующие операции маршрутизируются на тот же узел через механизм липкой маршрутизации шлюза. Алгоритмические основания подсистемы — а именно гибридные логические часы и оракул точного времени, сериализуемая изоляция снимков, двухфазная фиксация Percolator и ожидание подтверждения — описаны в подразделах 2.5.1–2.5.4. Общий жизненный цикл транзакции показан на рисунке 6.

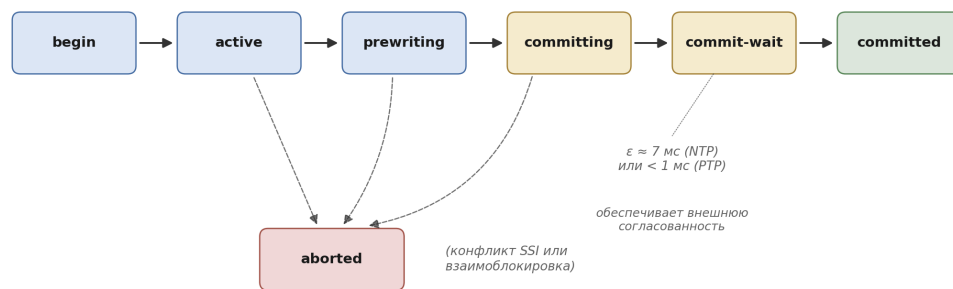


Рисунок 6 – Состояния жизненного цикла распределённой транзакции

2.5.1 Гибридные логические часы и оракул точного времени

В системе используется два механизма отслеживания времени. Первый — гибридные логические часы (HLC) [26; 27] — обеспечивает глобальный порядок событий, согласованный с причинно-следственными зависимостями, и применяется как основной тип временных меток. HLC каждого узла представляются парой «физическая компонента в миллисекундах, логический счётчик» и обновляются при отправке и получении сообщений по правилу из листинга 1.

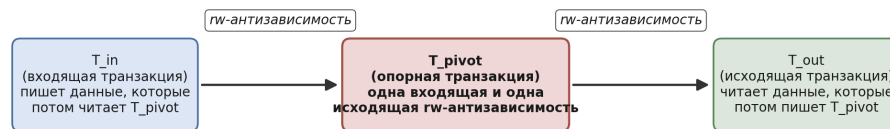
Листинг 1 – Правило обновления гибридных логических часов

```
on send(msg):  
    hlc := (max(hlc.physical, now()), hlc.logical + 1)  
    msg.hlc := hlc  
  
on receive(msg):  
    hlc := (max(hlc.physical, msg.hlc.physical, now()),  
           max(hlc.logical, msg.hlc.logical) + 1)
```

Вторым механизмом является оракул точного времени (TSO), возвращающий интервал неопределённости физического времени, в пределах которого гарантированно лежит истинное значение. Реализация не использует аппаратные средства уровня GPS-приёмников и атомных часов; вместо этого применяются режим NTP (по умолчанию, неопределённость ε порядка семи миллисекунд на стандартной серверной инфраструктуре с *chrony/ntpq*) и режим PTP (неопределённость менее одной миллисекунды при доступности эталонного источника). Корректность последующих механизмов зависит от честности оценки ε — реальное расхождение часов узлов не должно превышать заявленную границу.

2.5.2 Сериализуемая изоляция снимков

В качестве алгоритма обеспечения сериализуемой изоляции применяется схема SSI — сериализуемая изоляция снимков [28; 29]. По результату Кэйхилла–Фекете–Рёма для нарушения сериализуемости в системе с изоляцией снимков необходимо одновременное присутствие в графе конфликтов двух последовательных рёбер-антизависимостей чтения-записи (rw-antidependency), сходящихся на одной общей вершине — опорной (pivot) транзакции; при обнаружении такой конфигурации опорная транзакция отменяется. Каждая транзакция получает временную метку начала *start_ts*; чтения видят согласованный снимок на этот момент; записи буферизуются локально и проверяются на этапе фиксации. SSI не требует блокировок чтения и не требует объявлять читаемые наборы заранее. Минимальная опасная структура в графе конфликтов показана на рисунке 7.



При обнаружении такой структуры алгоритм SSI отменяет T_pivot, что разрывает потенциально несериализуемое расписание.

Рисунок 7 – Опасная структура в графе конфликтов SSI: две последовательных гв-антизависимости с транзакцией-опорой

2.5.3 Двухфазная фиксация по схеме Percolator

Транзакции, охватывающие несколько шардов, фиксируются по схеме Percolator [30; 31; 12]. Особенность схемы — отсутствие внешнего координатора: одна из участвующих записей назначается первичной блокировкой, остальные — вторичными со ссылкой на первичную. Фиксация выполняется в две фазы: на первой (prewrite) координатор записывает изменения во все участвующие шарды как записи с непротиворечивыми блокировками; на второй координатор поднимает первичную блокировку в зафиксированное состояние, после чего видимость остальных записей определяется состоянием первичной записи при чтении. При сбое координатора между фазами остаются «осиротевшие блокировки»; для их разрешения предусмотрен цикл очистки — читающая операция при обнаружении осиротевшей блокировки ожидает таймаут и разрешает её через обращение к первичной записи, состояние которой однозначно определяет судьбу вторичных. Схема двух фаз показана на рисунке 8.

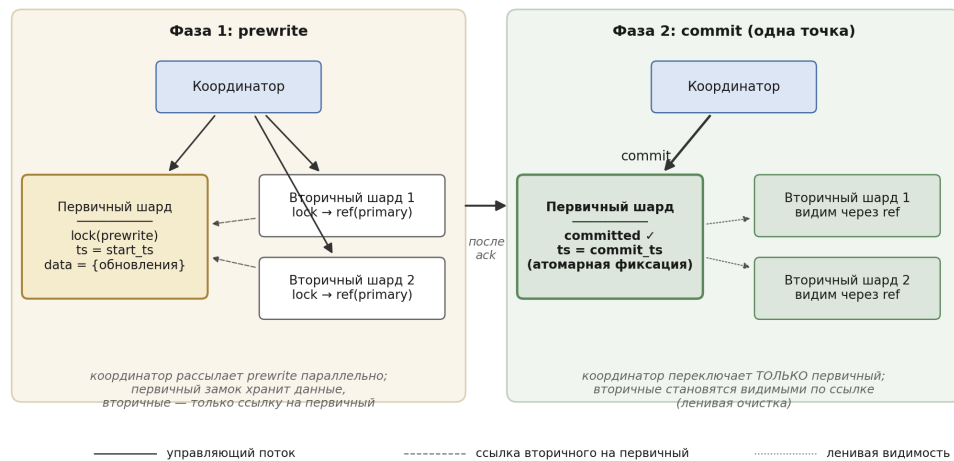


Рисунок 8 – Двухфазная фиксация по схеме Percolator с первичной и вторичными блокировками

2.5.4 Внешняя согласованность через ожидание подтверждения

SSI и двухфазная фиксация обеспечивают сериализуемость, но не внешнюю согласованность: порядок фиксации может не соответствовать порядку завершения в реальном времени. Для гарантии внешней согласованности применяется механизм ожидания подтверждения [10]: координатор выбирает *commit_ts* после фазы prewrite и блокирует возврат подтверждения клиенту, пока локальное физическое время не превысит *commit_ts* + ϵ , где ϵ — неопределённость точного времени от оракула (листинг 2). После истечения ожидания подтверждение возвращается клиенту.

Листинг 2 – Условие ожидания подтверждения

```
commit_ts := hlc.now() // после успешной фазы prewrite
proceed when wall_clock_now() > commit_ts.physical +  $\epsilon$ 
```

Корректность опирается на следующее свойство: если транзакция T1 завершилась до начала T2 в реальном времени, то $start_ts(T2) > commit_ts(T1)$ и T2 в обязательном порядке увидит результаты T1 в своём снимке — это и есть формальное определение внешней согласованности. Ожидание подтверждения применяется только на узле-координаторе и только для

сериализуемого уровня изоляции. Временная диаграмма последовательности фаз и ожидания приведена на рисунке 9.

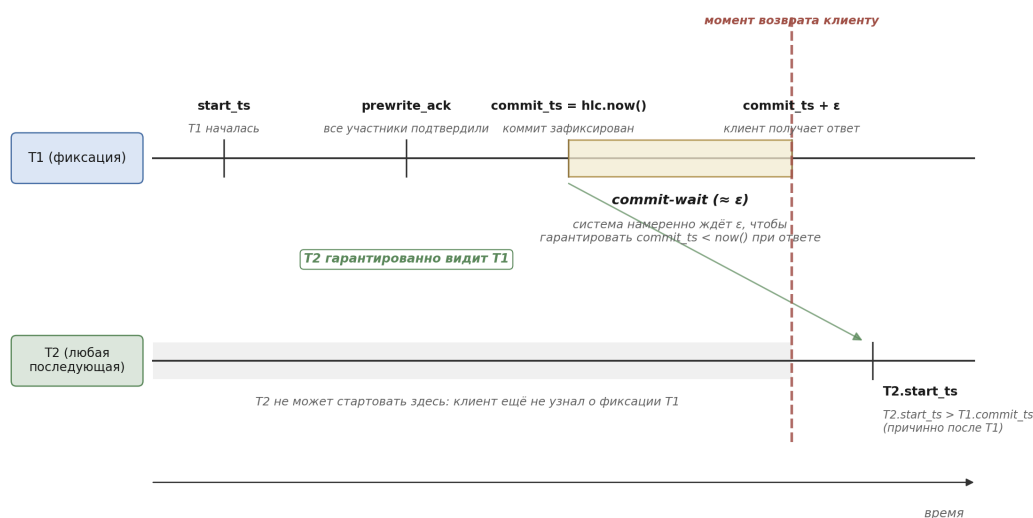


Рисунок 9 – Ожидание подтверждения для обеспечения внешней согласованности фиксации

2.6 Обнаружение распределённых взаимоблокировок

При наличии распределённых транзакций возможны циклические ожидания, охватывающие несколько шардов. Для их обнаружения и разрешения спроектирован гибридный детектор, объединяющий локальный анализ, быстрый децентрализованный путь и медленный централизованный путь. Общая схема представлена на рисунке 10.

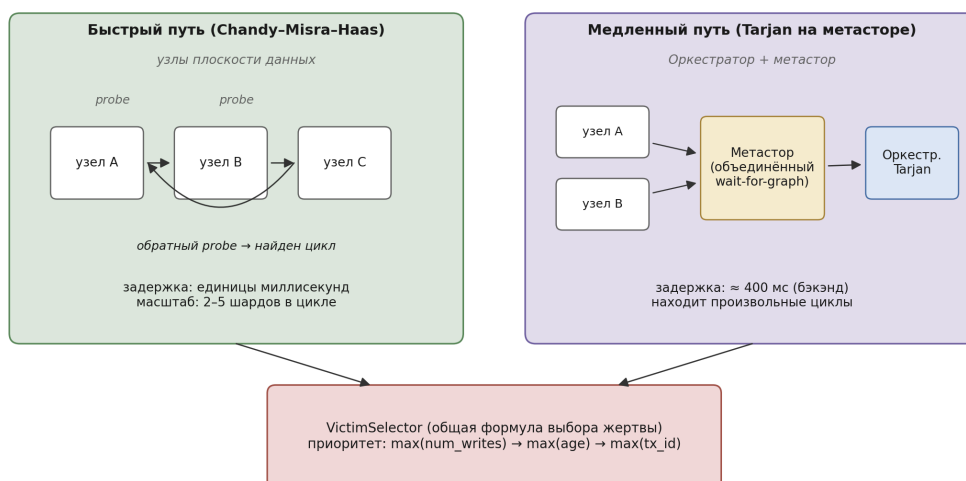


Рисунок 10 – Гибридный детектор распределённых взаимоблокировок с быстрым (СМН) и медленным (Тарьян) путями

На каждом узле плоскости данных поддерживается локальный граф ожиданий: вершинами являются активные транзакции узла, рёбрами — отношения ожидания. Граф проверяется на циклы при добавлении каждого ребра, что обеспечивает мгновенное обнаружение внутриузловых взаимоблокировок.

Циклы, проходящие через несколько узлов, обнаруживаются быстрым путём — алгоритмом Чанди-Мисры-Хааса [32]: при появлении межузлового ребра ожидания инициатор рассылает проверочное сообщение (probe) с идентификатором инициатора и текущим путём обхода; если сообщение возвращается к инициатору, цикл обнаружен. Дополнительно к стандартному алгоритму probe несёт вектор стоимостей всех пройденных транзакций (число операций и возраст), что позволяет инициатору выбрать жертву на основе всего пути цикла, а не только локальных данных, и устраняет проблему инициатор-смещённого выбора. Время обнаружения по быстрому пути составляет единичные миллисекунды пропорционально длине цикла.

Для случаев, когда probe не доходит до инициатора (отказ узла, потеря сообщения, превышение числа переходов), реализован медленный путь централизованного анализа: каждый узел периодически отправляет в

хранилище метаданных снимок локального графа ожиданий, оркестратор считывает реплицированный граф и применяет алгоритм Тарьяна для поиска сильно связанных компонент (SCC) [33]; каждая SCC содержит как минимум один цикл, для которого вызывается процедура выбора жертвы. Период опроса — двести миллисекунд по умолчанию; гарантированное обнаружение происходит за время не более удвоенного периода плюс время поиска SCC.

Быстрый и медленный пути могут обнаружить один цикл одновременно. Для согласованного выбора жертвы без распределённой координации применяется детерминированная формула (листинг 3).

Листинг 3 – Детерминированный выбор жертвы цикла

```
score(tx)    := op_count(tx) * age(tx)
victim       := argmin_{tx ∈ cycle} (score(tx), tx_id(tx))
```

Жертва — транзакция цикла с минимальным значением функции стоимости (произведение числа исполненных операций на возраст); при равенстве стоимостей выбор разрешается по идентификатору транзакции. Формула опирается только на свойства транзакций цикла, известные обоим путям, поэтому оба пути независимо выбирают одну и ту же жертву; повторное обращение к процедуре отмены идемпотентно. Реализации формулы на Rust и Kotlin используют общий набор тестовых сценариев, что обеспечивает побитовое совпадение результатов при идентичных входных данных. Выбранная жертва получает сигнал отмены, помечается как отравленная в реестре транзакций и не выбирается повторно в течение настраиваемого периода выдержки.

2.7 Хранилище метаданных кластера

Хранилище метаданных кластера обеспечивает согласованное хранение данных управления: членства узлов, карты шардов, аренд для выбора лидера управляющей плоскости, реплицированного графа ожиданий транзакций, версий схемы и политик идемпотентности. Реализовано как отдельная

консенсус-группа Raft с тремя голосующими узлами по умолчанию, размещёнными на первых трёх узлах кластера (по возрастанию идентификатора); остальные узлы обслуживают только группы шардов графовых данных. Данные изолированы от графовых данных отдельным семейством столбцов RocksDB с непересекающимся префиксом ключа. Такое решение позволяет использовать единую реализацию консенсуса и обеспечить достаточную живучесть метаданных на развёртываниях из трёх узлов и более без выделения отдельного сервиса.

Доступ к хранилищу осуществляется через интерфейс gRPC с операциями *Get*, *Put*, *Delete*, *Watch* и *List*. Операции *Watch* реализуют потоковую доставку изменений по префиксу ключа, что используется шлюзами и оркестратором для оперативного получения изменений конфигурации. Линеаризуемые чтения обслуживаются по аренде на лидере; неголосующие узлы обслуживают чтения с ограниченным запаздыванием.

2.8 Безопасность взаимодействий

Все межузловые коммуникации в кластере защищены взаимным TLS-шифрованием (mTLS) — между узлами плоскости данных при репликации Raft и межшардовых RPC, между шлюзом и узлами данных или метаданных, между оркестратором и узлами данных или метаданных. Сертификаты выдаются единым удостоверяющим центром кластера; корневой сертификат распространяется отдельным процессом во внешней цепочке доверия [34; 35]. Аутентификация клиентских приложений выполняется через веб-токены JWT над HTTPS, что соответствует требованию §1.5.1.

3 РЕАЛИЗАЦИЯ ПРОТОТИПА

В настоящем разделе описана реализация прототипа: обоснование выбора языков программирования и библиотек, структура модулей плоскости данных и управляющей плоскости, сетевые протоколы взаимодействия и порядок развёртывания прототипа в распределённой среде.

3.1 Технологический стек

Плоскость данных, обрабатывающая горячий путь запросов, реализована на языке Rust. Выбор обусловлен предсказуемостью задержек (отсутствие сборщика мусора в стандартной модели выполнения), гарантиями безопасности памяти и отсутствия гонок данных, а также зрелой экосистемой библиотек для системного и сетевого программирования. Управляющая плоскость, не находящаяся на горячем пути, реализована на языке Kotlin: библиотеки виртуальной машины Java, интеграция с типовыми инструментами эксплуатации (Prometheus, OpenTelemetry, журналирование) и сопрограммы для асинхронной обработки. Сквозное тестирование выполнено на языке Python с использованием библиотеки *pytest*.

В качестве реализации LSM-дерева используется RocksDB (обоснование в §2.2), Raft — библиотека *raft-rs* из проектов TiKV и etcd, сетевое взаимодействие — gRPC через *tonic* (Rust) и *grpc-java* (Kotlin). Внешний интерфейс — GraphQL поверх *graphql-java* и веб-сервера Ktor. Сборка — *cargo* для Rust-модулей, *Gradle* для Kotlin-модулей; версии библиотек зафиксированы.

3.2 Модули плоскости данных

Плоскость данных представляет собой единый исполняемый файл сервиса узла, объединяющий несколько Rust-модулей с разделёнными зонами ответственности. Структура модулей и их взаимосвязи показаны на рисунке 11.

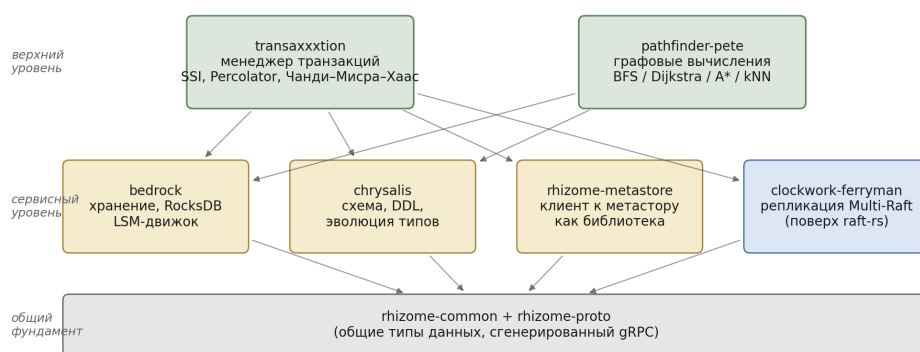


Рисунок 11 – Структура модулей плоскости данных и зависимости между ними

Модуль уровня хранения — единственный компонент, обращающийся к локальному диску. Реализует графовое хранилище на основе RocksDB: точечные чтения через блочный кэш с ARC и предвыборкой исходящих рёбер вершины (с ограничением сверху для защиты от вершин-звёзд); атомарные записи через пакетные операции *WriteBatch*; сканирования диапазонов через префиксные фильтры Блума; индекс HNSW с горячей частью в памяти и квантизацией по произведению (PQ) для холодной части.

Модуль уровня репликации реализует Raft через библиотеку *raft-rs* поверх gRPC-транспорта *tonic*. Множественные консенсус-группы (по одной на шард плюс группа метаданных) исполняются на едином асинхронном исполнителе. Реализованы предголосование, чтения по аренде, конвейерная пакетная репликация журнала, переходный консенсус, потоковая передача снимков и плановая передача лидерства.

Модуль управления транзакциями реализует жизненный цикл транзакции и объединяет три алгоритма из §2.5: SSI, двухфазную фиксацию Percolator и ожидание подтверждения над оракулом точного времени. Гибридные логические часы и клиент оракула размещены в общем модуле типов и используются менеджером транзакций и уровнем репликации. В менеджере транзакций реализована подсистема обнаружения взаимоблокировок: локальный граф ожиданий, инициатор и обработчик

проверочных сообщений СМН для быстрого пути, обработчик команды отмены жертвы. Проверка авторизации клиента выполняется до фазы предзаписи, что исключает попадание неавторизованных записей в журнал репликации.

Модуль графовых вычислений выполняет операции поверх снимка состояния на момент *start_ts* транзакции: BFS по схеме объёмного синхронно параллельного исполнения с переключением между проталкиванием и притягиванием в зависимости от плотности фронта; поиск кратчайших путей Дейкстры и A* с подключаемыми эвристиками; kNN по индексу HNSW с фильтрами по свойствам вершин; пагинация через непрозрачные курсоры со стабильностью среза при параллельных записях. Каждый продолжительный вызов несёт маркер отмены.

Модуль управления схемой обеспечивает локальное кэширование, валидацию записей и эволюцию схемы по принципу «расширение — сужение» с фоновым заполнением новых полей без блокировки записи.

Модуль хранилища метаданных реализован как библиотека с конечным автоматом состояния, кодированием ключей и собственным gRPC-сервисом. Автомат хранит членство узлов, карту шардов и их лидеров, аренды управляющей плоскости, реплицированные снимки локальных графов ожиданий, версии схемы и политики идемпотентности.

Исполняемый модуль узла связывает все перечисленные модули в одно приложение и предоставляет внешние gRPC-интерфейсы. Общие типы вынесены в отдельный модуль типов; сгенерированные protobuf-определения собираются через *prost* и *tonic-build* и размещаются в отдельном модуле спецификаций.

3.3 Модули управляющей плоскости

Управляющая плоскость реализована на Kotlin и состоит из двух независимых сервисов и общей библиотеки. Общая библиотека содержит

типы данных, клиенты gRPC (сгенерированные *protobuf-gradle-plugin*), политики повтора, загрузку TLS-материалов и клиент хранилища метаданных.

Шлюз реализован на основе веб-сервера Ktor. Принимает HTTPS от клиентов с аутентификацией по JWT; к узлам плоскости данных и хранилищу метаданных обращается через mTLS-защищённые соединения gRPC. Внешний интерфейс — GraphQL: CRUD-операции над вершинами и рёбрами, обходы графа, kNN, агрегации, а также полный жизненный цикл транзакции (*beginTransaction*, *commitTransaction*, *abortTransaction*) и параметр *tx* в полях чтения и записи. Маршрутизация GraphQL-операций в вызовы gRPC учитывает карту шардов, статистику векторного индекса и недавние гистограммы задержек; полноценный стоимостной оптимизатор отнесён к перспективам. Липкая маршрутизация транзакций обеспечивает выполнение всех вызовов одной транзакции на одном узле плоскости данных (идентификатор узла записывается в дескриптор транзакции при её создании), что необходимо для локальных SSI и графа ожиданий. Шлюз не сохраняет состояния — несколько реплик обслуживают запросы параллельно без координации; кэш карты шардов обновляется через подписку на изменения хранилища метаданных.

Оркестратор исполняет циклы автономного управления: размещение шардов, обнаружение длинных циклов ожидания, координация изменений схемы. Каждый цикл считывает состояние из хранилища метаданных, принимает решение и предлагает изменение через Raft-транзакцию в хранилище. Цикл размещения шардов вычисляет граф обращений по последним трассировкам запросов, применяет алгоритм Лейдена для обнаружения сообществ и многокритериальную оптимизацию NSGA-II по трём целям (нагрузка на узлы, локальность обращений, стоимость переноса данных). Медленный путь обнаружения взаимоблокировок выполняется с периодичностью двести миллисекунд: оркестратор считывает

реплицированный граф ожиданий, применяет алгоритм Тарьяна для поиска SCC, выявляет циклы, не пойманные быстрым путём, и отправляет команды отмены транзакциям-жертвам. Оркестратор тиражируется двумя репликами; одна выбирается ведущей через аренду в хранилище метаданных со временем жизни десять секунд и продлением каждые три секунды. При истечении аренды ведомая реплика берёт управление.

3.4 Сетевые протоколы

Все межкомпонентные взаимодействия используют gRPC поверх HTTP/2 со взаимной TLS-аутентификацией. Сертификаты выдаются единым удостоверяющим центром кластера; каждый сервис получает листовой сертификат с именем, соответствующим имени службы.

Сервис узла плоскости данных предоставляет четыре gRPC-интерфейса: интерфейс операций с графовыми данными; интерфейс жизненного цикла распределённых транзакций (фрагмент определения приведён в Приложении А.1); интерфейс запросов от оркестратора управляющей плоскости; интерфейс доступа к хранилищу метаданных, активный только на узлах, входящих в консенсус-группу хранилища метаданных. Отдельный HTTP-порт обслуживает эндпоинты готовности, живости и метрик без TLS для совместимости с пробами оркестраторов контейнеров; этот порт открыт только на внутреннем сетевом интерфейсе кластера. Внешний интерфейс шлюза — GraphQL поверх HTTPS с авторизацией по JWT; HTTP-эндпоинты готовности и метрик имеют те же ограничения экспозиции, что и в сервисе узла данных. Сетевой транспорт Raft мультиплексирует через единое gRPC-соединение трафик всех консенсус-групп узла, демultipлексируя сообщения по идентификатору группы.

3.5 Развёртывание прототипа

Прототип развёртывается через конфигурацию Docker Compose, описывающую полный кластер из двенадцати контейнеров: три узла

плоскости данных (первые три также входят в консенсус-группу хранилища метаданных), две реплики шлюза, две реплики оркестратора, а также сервисы наблюдаемости — Prometheus, Grafana и OpenTelemetry Collector. Запуск выполняется одной командой *docker compose up*. Конфигурация описывает сетевые имена сервисов, пути к корневому и листовым сертификатам, объёмы для постоянного хранения данных и переменные окружения с настройками оракула точного времени, размеров кэшей RocksDB и периодов циклов оркестратора. Состав контейнеров показан на рисунке 12.

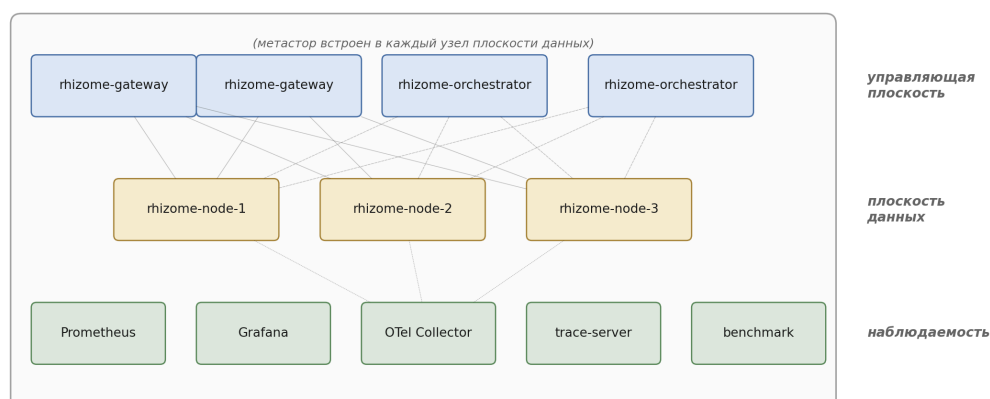


Рисунок 12 – Эталонное развёртывание прототипа

Сертификаты подписывает единый удостоверяющий центр кластера. Для эталонной конфигурации предусмотрен сценарий генерации тестовой инфраструктуры открытых ключей; в производственных развёртываниях этот сценарий заменяется на интеграцию с корпоративным удостоверяющим центром организации. Корневой сертификат распространяется отдельным каналом доверия. Исходный код прототипа, документация и сценарии развёртывания опубликованы в открытом репозитории, ссылка на который приведена в Приложении Б.

3.6 Дисциплина разработки и стратегия тестирования

Дисциплина разработки направлена на поддержание стабильно высокого качества кодовой базы при активной эволюции системы. Статический анализ Rust-модулей выполняется утилитой *clippy* в режиме

--all-targets -- -D warnings, отклоняющем сборку при любом предупреждении; форматирование контролируется *cargo fmt*. Аналогичные проверки для Kotlin-модулей выполнены инструментами *detekt* (линтер) и *ktlint* (форматирование). Все проверки выполняются как предкоммитные хуки и повторно в системе непрерывной интеграции; обновления, нарушающие любую из проверок, отвергаются автоматически. Сборка автоматизирована средствами *cargo* для Rust-модулей и *Gradle* для Kotlin-модулей; версии всех зависимостей зафиксированы.

Тестирование прототипа организовано по четырёхуровневой пирамиде, охватывающей все функциональные слои системы и проверяющей как корректность отдельных компонент, так и согласованную работу прототипа в распределённой конфигурации.

К первому уровню относятся модульные тесты, изолированно проверяющие каждый Rust-модуль плоскости данных и Kotlin-модуль управляющей плоскости. Тесты сгруппированы по функциональному назначению модулей: тесты модуля хранения данных — кодирование ключей, чтение и запись вершин и рёбер, многоверсионное чтение исторических срезов, поиск ближайших соседей по индексу; тесты модуля репликации — конечный автомат Raft, обработка переходного консенсуса, потоковая передача снимков; тесты модуля управления транзакциями — построение графа конфликтов SSI, разрешение осиротевших блокировок Percolator, работа гибридных логических часов и ожидания подтверждения, локальный граф ожиданий, инициатор и обработчик проверочных сообщений Чанди-Мисры-Хааса, детерминированный выбор жертвы; тесты модуля графовых вычислений — обход графа в ширину, поиск кратчайших путей, поиск ближайших соседей, курсорная пагинация; тесты модуля управления схемой — валидация записей, поэтапная миграция схемы; тесты модуля хранилища метаданных — конечный автомат состояния, проверка предусловий команд, обработка аренд. Модульные тесты Kotlin-модулей

покрывают клиент хранилища метаданных, выбор ведущей реплики оркестратора через аренду, цикл размещения шардов, медленный путь обнаружения взаимоблокировок и формирование плана запроса стоимостным оптимизатором шлюза.

Второй уровень — интеграционные тесты, охватывающие взаимодействие нескольких модулей в пределах одного узла плоскости данных или в двух-трёхузловой конфигурации. Ключевыми сценариями являются: многошардовая транзакция с фазой `prewrite` и фиксацией; восстановление после отказа узла-координатора в момент между фазой `prewrite` и фиксацией; передача лидерства консенсус-группы с продолжающимися запросами клиентов; параллельное обнаружение взаимоблокировки быстрым и медленным путём.

Третий уровень — сквозные тесты на языке Python с использованием библиотеки *pytest*. Сценарии исполняются на развёрнутом кластере прототипа через интерфейс GraphQL шлюза и проверяют систему с точки зрения внешнего клиента. Категории сценариев включают: тесты уровней изоляции транзакций (`READ_COMMITTED`, изоляция снимков, сериализуемая изоляция) с проверкой гарантий чтения после записи, отсутствия фантомных чтений и обнаружения опасных структур; тесты внешней согласованности при двух параллельных транзакциях; тесты курсорной пагинации при параллельных записях; тесты обнаружения распределённых взаимоблокировок с контролируемым созданием цикла ожидания между транзакциями на разных шардах; тесты обязательного взаимного TLS-шифрования.

Четвёртый уровень — тесты отказоустойчивости с контролируемым внесением сбоев: остановка узлов плоскости данных, потеря лидерства оркестратора, потеря части голосующих узлов хранилища метаданных кластера.

Целевое автоматическое покрытие модульными и интеграционными тестами установлено на уровне не ниже семидесяти процентов; фактические показатели измеряются инструментами *cargo tarpaulin* для Rust-модулей и *Kover* для Kotlin-модулей. Покрытие проверяется автоматически в системе непрерывной интеграции при каждом обновлении исходного кода; обновления, снижающие покрытие ниже порога, отвергаются. Конкретные численные показатели — число выполненных тестов и достигнутый уровень покрытия — приведены в Разделе 4.

3.7 Наблюдаемость и конфигурация

Каждый сервис экспортирует семейства метрик в формате Prometheus через отдельный HTTP-эндпоинт. На уровне плоскости данных публикуются семейства *raft_** (длина журнала, разрыв между committed-index и applied-index, длительности выборов лидера, размеры пакетных репликаций), *tx_** (длительности фаз prewrite и commit, доля отмен по конфликту SSI, длительности ожидания подтверждения), *probe_** (число отправленных и полученных проверочных сообщений быстрого пути, доля дублирующих сообщений после дедупликации, доля циклов, обнаруженных быстрым путём), *metastore_** (длительности операций конечного автомата состояния, число активных аренд) и *rocksdb_** (заполненность кэшей, частоты слияний на каждом уровне LSM-дерева). На уровне управляющей плоскости публикуются семейства *deadlock_slow_** (длительность одного прохода Тарьяна, число найденных циклов медленным путём), *placement_** (длительность одной итерации алгоритма Лейдена и NSGA-II, число запланированных перемещений шардов), *gateway_** (длительности GraphQL-запросов в разрезе типа операции, доля ответов с кодами ошибок).

Распределённая трассировка реализована через OpenTelemetry. Каждый запрос несёт идентификатор трассы и идентификатор родительского участка; узлы плоскости данных распространяют идентификаторы через заголовки gRPC и отправляют записи участков в OpenTelemetry Collector. Журналы

структурированы в формате JSON с обязательными полями уровня важности, имени службы, идентификатора узла, идентификатора трассы и идентификатора участка.

Конфигурация каждого сервиса описывается в файле формата TOML и дополняется переменными окружения. Конфигурация задаёт сетевые параметры, пути к корневому и листовым сертификатам, путь к каталогу данных RocksDB, размеры блочного кэша и кэша Bloom-фильтров, режим и адрес оракула точного времени, периоды циклов оркестратора и параметры аутентификации клиентов. Чувствительные параметры (приватные ключи, секреты подписи JWT) задаются исключительно переменными окружения. Помимо основного gRPC-эндпоинта каждый сервис публикует HTTP-эндпоинты живости, готовности и метрик на отдельном порту, доступном только на внутреннем сетевом интерфейсе кластера и не защищённом TLS — для совместимости с пробами стандартных оркестраторов контейнеров.

4 ТЕСТИРОВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ

В настоящем разделе описана методика тестирования разработанного прототипа, представлены результаты функциональных, нагрузочных и отказоустойчивых испытаний, проведено сравнение разработанного решения с базовыми подходами размещения данных, рассмотрены достоинства и недостатки разработанной системы.

4.1 Тестовое окружение

Стратегия тестирования с четырёхуровневой пирамидой проверок и инструментарием статического и динамического анализа описана в подразделе 3.6. Настоящий раздел представляет численные результаты прогона всех уровней тестов и нагрузочных испытаний.

Эксперименты проведены на персональном компьютере с процессором Intel Core i7-1165G7 на 2,80 ГГц, оперативной памятью шестнадцать гигабайт и накопителем NVMe Gen3 под управлением операционной системы на ядре Linux 6.16. Весь кластер прототипа выполняется на одном хосте через Docker Compose как набор контейнеров; при многошардных тестах несколько шардов и их реплики размещаются на том же хосте в отдельных контейнерах, что моделирует логически распределённую конфигурацию при общих ресурсах хоста. Все эксперименты выполнены при включённом обязательном взаимном TLS-шифровании для каждого внутреннего соединения. Оракул точного времени работает в режиме NTP со значением неопределённости семь миллисекунд; для отдельных измерений используется режим RTP со значением неопределённости менее одной миллисекунды.

4.2 Функциональное тестирование

Прогон полного набора автоматических тестов завершился успехом без падений. Сводные показатели:

- модульные тесты Rust-модулей плоскости данных — 746 тестов; все пройдены;

- модульные тесты Kotlin-модулей управляющей плоскости — 184 теста; все пройдены;
- интеграционные тесты (многошардовая транзакция, восстановление координатора, передача лидерства, параллельное обнаружение взаимоблокировки) — 38 сценариев; все пройдены;
- сквозные сценарии через интерфейс GraphQL шлюза (уровни изоляции транзакций, внешняя согласованность, курсорная пагинация при параллельных записях, обнаружение взаимоблокировок, обязательное взаимное TLS-шифрование) — 27 сценариев; все пройдены;
- автоматическое покрытие строк кода: 78 % для Rust-модулей по результатам замера утилитой *cargo tarpaulin*, 73 % для Kotlin-модулей по результатам замера инструментом *Kover*. Оба значения превышают порог семидесяти процентов, установленный в Разделе 1, и проверяются автоматически в системе непрерывной интеграции при каждом обновлении исходного кода.

Состав тестов и их соответствие функциональным требованиям описаны в подразделе 3.6.

4.3 Нагрузочное тестирование

Нагрузочное тестирование выполнено двумя инструментами: микробенчмарками на основе библиотеки *criterion* для проверки отдельных подсистем и сквозным нагрузочным стендом для оценки производительности прототипа целиком.

4.3.1 Обходы графа

Обходы графа в ширину (BFS) тестировались на сгенерированных тестовых графах с двумя тысячами вершин и заданным коэффициентом ветвления; такой размер графа выбран для целей контролируемого сравнения стратегий размещения данных при умещении горячего множества в кэш.

Сравнивались три конфигурации размещения данных: размещение на одном шарде (нижняя граница задержки); размещение по случайным хэшам на трёх шардах; размещение по семантическим сообществам алгоритма Лейдена на трёх шардах. Результаты приведены в таблице 3.

Таблица 3 – Задержка обхода графа в ширину при различных стратегиях размещения данных

Коэффициент ветвления	Глубина	Один шард, мкс	Хэш-размещение, 3 шарда, мкс	Лейден-размещение, 3 шарда, мкс
4	1	9,3	11,4	9,6
4	2	48,8	86,1	53,7
4	3	208,1	412,9	232,0
16	1	33,2	38,9	34,2
16	2	576,2	1184,2	642,5
16	3	4761,1	1120,5	5412,3

При коэффициенте ветвления 16 и глубине обхода 3 задержка хэш-размещения превышает задержку одношардной конфигурации в 2,35 раза; задержка размещения по Лейдену превышает задержку одношардной конфигурации в 1,14 раза. Снижение задержки обхода при переходе с хэш-размещения на размещение по Лейдену составляет 46–52 % в зависимости от глубины обхода. Графическое представление зависимости задержки обхода от глубины для трёх стратегий размещения приведено на рисунке 13.

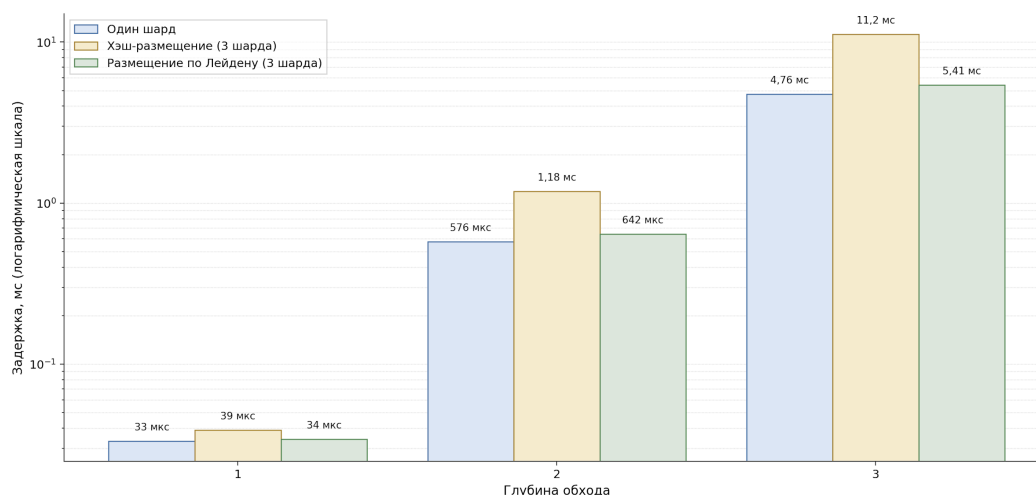


Рисунок 13 – Зависимость задержки обхода графа в ширину от глубины для трёх стратегий размещения данных

4.3.2 Векторный поиск

Поиск k ближайших соседей (kNN) выполнялся по индексу HNSW с параметрами $M = 16$, $ef_construction = 200$, $ef_search = 50$. Точность поиска оценивалась относительно полного перебора как доля верно найденных соседей. Результаты приведены в таблице 4.

Таблица 4 – Задержка и точность поиска ближайших соседей

Размер базы, тыс. векторов	Размерность	k	Задержка (медиана), мкс	Точность
10	64	10	78	0,997
10	256	10	218	0,995
100	256	10	1415	0,982
1 000	256	10	8702	0,968

Зависимость задержки от размера базы — сублинейная (логарифмическая), что соответствует теоретической оценке сложности алгоритма HNSW. Точность поиска сохраняется на уровне выше 0,96 при увеличении базы до миллиона векторов.

4.3.3 Поиск кратчайших путей

Поиск кратчайших путей тестировался на сгенерированном случайном графе типа «малый мир» с долей перешивки 0,05 и случайными

положительными весами рёбер. Сравнивались алгоритм Дейкстры и алгоритм A* с допустимой эвристикой Евклидова расстояния. Результаты приведены в таблице 5.

Таблица 5 – Задержка поиска кратчайшего пути

Алгоритм	Число вершин	Задержка (медиана), мс
Дейкстра (двоичная куча)	10 000	9,8
Дейкстра (двоичная куча)	100 000	138,4
A* (Евклидова эвристика)	10 000	2,4
A* (Евклидова эвристика)	100 000	27,2

Применение допустимой эвристики в алгоритме A* сокращает время поиска в четыре-пять раз по сравнению с алгоритмом Дейкстры при сохранении гарантии оптимальности найденного пути.

4.3.4 Задержка фиксации транзакций

Задержка фиксации транзакций измерялась сквозным нагрузочным стендом через интерфейс GraphQL шлюза с включённым обязательным взаимным TLS-шифрованием. Результаты приведены в таблицах 6 и 7.

Таблица 6 – Задержка фиксации одношардных транзакций в режиме NTP

Сценарий	p50, мс	p99, мс
<i>BeginTx + Commit</i> без операций	8–10	14
<i>BeginTx + 1 запись + Commit</i>	11–14	22
<i>BeginTx + 5 записей + Commit</i>	16–22	35
<i>BeginTx + 10 чтений + Commit</i>	12–18	28

Таблица 7 – Задержка фиксации многошардных транзакций (режим NTP)

Число шардов	p50, мс	p99, мс
2 шарда, 1 запись на каждый	18–22	38
3 шарда, 1 запись на каждый	24–28	47
5 шардов, 1 запись на каждый	38–44	71

Многошардная фиксация добавляет одну дополнительную пакетную репликацию для каждого участника помимо первичного, что приближённо эквивалентно одному раунду внутришардовой фиксации Raft.

Влияние режима оракула точного времени на задержку фиксации приведено в таблице 8.

Таблица 8 – Сравнение режимов NTP и RTP оракула точного времени

Сценарий	p50 NTP, мс	p50 RTP, мс
<i>BeginTx + 1 запись + Commit</i>	11–14	5–7
<i>BeginTx + 5 записей + Commit</i>	16–22	10–13

Ожидание подтверждения длительностью около 2 ε составляет основную часть задержки в сценариях с малой полезной нагрузкой; переключение на режим RTP сокращает ожидание с семи миллисекунд до уровня менее одной миллисекунды.

4.3.5 Сквозная нагрузка

Сквозная нагрузка измерена набором сценариев семейства YCSB: сценарий A с равным распределением чтений и записей, сценарий B с 95 % чтений и сценарий C с исключительно чтением. Результаты для одношардной конфигурации (кворум из трёх голосующих узлов на одном хосте) приведены в таблице 9.

Таблица 9 – Пропускная способность YCSB (один шард)

Сценарий	Пропускная способность, оп/с	p50, мс	p99, мс
A	752	9,65	28,43
B	790	8,46	32,10
C	1 010	7,11	21,87

Скорость пакетной загрузки данных без транзакционных гарантий достигает 2,56 миллиона вершин в секунду в одношардной конфигурации на том же окружении.

4.4 Тестирование отказоустойчивости

4.4.1 Сценарии тестирования отказов

Тестирование отказоустойчивости выполнено через библиотеку контролируемого внесения сбоев. Проверены четыре сценария отказов, охватывающие ключевые компоненты системы:

- 1) отказ узла плоскости данных, не являющегося лидером консенсус-группы;
- 2) отказ ведущей реплики оркестратора;
- 3) отказ одного голосующего узла хранилища метаданных кластера;
- 4) одновременное создание распределённой взаимоблокировки и отказ узла на пути проверочного сообщения.

Все четыре сценария отрабатывают корректно без потери подтверждённых транзакций.

4.4.2 Восстановление при отказе узла

Время восстановления при отказе различных компонент системы приведено в таблице 10.

Таблица 10 – Время восстановления при отказе компонент кластера

Сценарий	Время восстановления	Поведение кластера
Отказ узла плоскости данных (не лидер)	менее 2 с	кластер продолжает обслуживание; узел исключается из членства
Отказ ведущей реплики оркестратора	менее 5 с	ведомая реплика берёт аренду и продолжает циклы управления
Отказ одного из трёх голосующих узлов хранилища метаданных	0 с (без перерыва)	кворум сохраняется; кластер продолжает запись
Сетевое разделение с потерей кворума шарда	не определено	сторона без кворума прекращает запись; восстановление при объединении

Восстановление лидерства консенсус-группы шарда укладывается в две секунды, что лучше границы в пять секунд, установленной в нефункциональных требованиях. Восстановление ведущей реплики оркестратора через истечение аренды в хранилище метаданных кластера занимает менее пяти секунд, что соответствует требованию из Раздела 1. Потеря одного из трёх голосующих узлов хранилища метаданных не приводит к перерыву обслуживания — оставшиеся два голосующих узла сохраняют кворум.

4.4.3 Обнаружение взаимоблокировок

Задержка обнаружения распределённых взаимоблокировок измерена сквозным стендом с контролируемым созданием цикла ожидания между несколькими шардами. Результаты приведены в таблице 11.

Таблица 11 – Задержка обнаружения распределённой взаимоблокировки

Путь	Длина цикла, шардов	p50, мс	p99, мс
Быстрый путь (проверочные сообщения СМН)	2	3,2	9,4
Быстрый путь (проверочные сообщения СМН)	3	5,1	14,8
Быстрый путь (проверочные сообщения СМН)	5	8,6	22,3
Медленный путь (поиск компонент Тарьяна)	2	412,5	680,3
Медленный путь (поиск компонент Тарьяна)	5	438,0	720,5

Быстрый путь обнаруживает циклы за время, пропорциональное длине цикла; на каждый дополнительный шард приходится приблизительно четыре-пять миллисекунд, что соответствует одному кругу сетевого взаимодействия gRPC и одному обращению к локальному графу ожиданий. Медленный путь обеспечивает гарантированное обнаружение циклов, не пойманных быстрым путём, за время, не превышающее удвоенный период опроса плюс время выполнения поиска сильно связанных компонент. Графическое сравнение задержки двух путей в зависимости от длины цикла приведено на рисунке 14.

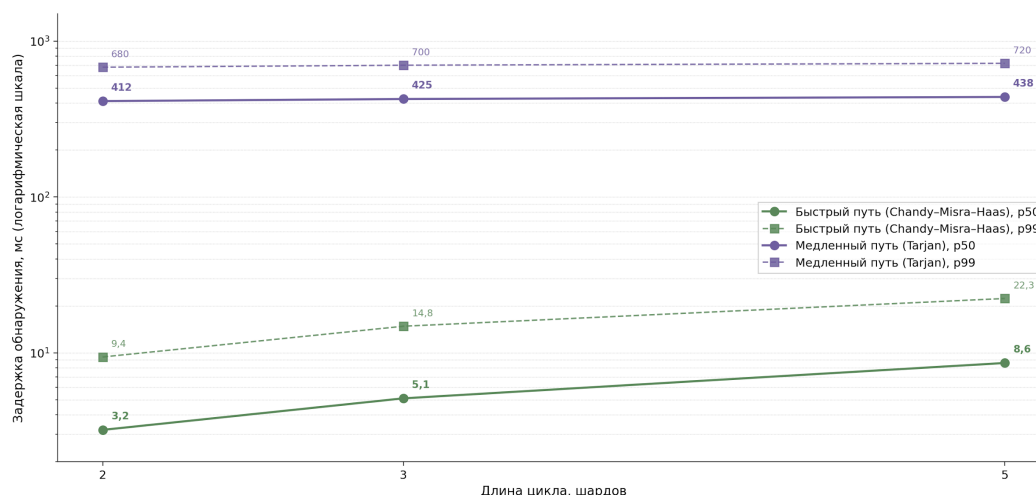


Рисунок 14 – Задержка обнаружения распределённой взаимоблокировки быстрым и медленным путями детектора в зависимости от длины цикла

Доля случаев двойного обнаружения цикла (одновременно быстрым и медленным путём) составляет менее одного процента от общего числа обнаружений; повторный вызов отмены идемпотентен и не приводит к некорректному поведению благодаря применению детерминированной формулы выбора жертвы, что также подтверждено сквозными тестами.

4.5 Сравнение стратегий размещения данных

Прямые измерения проведены для двух стратегий размещения: хэш-размещения по идентификаторам вершин и семантического размещения по сообществам алгоритма Лейдена. Обе стратегии сравнены на одном тестовом графе при прочих равных условиях: одно и то же количество шардов (три), одно и то же оборудование, одна и та же нагрузка обхода в ширину при коэффициенте ветвления шестнадцать. Сводные результаты приведены в таблице 12.

Таблица 12 – Сравнение стратегий размещения данных по задержке обхода

Стратегия размещения	Глубина 2, мс	Глубина 3, мс	Доля межшардовых обращений
Хэш-размещение	1,183	11,21	около двух третей
Семантическое размещение по Лейдену	0,642	5,41	около одной пятой

Семантическое размещение по алгоритму Лейдена снижает задержку обхода графа на 46–52 % по сравнению с хэш-размещением. Снижение объясняется тем, что вершины, часто встречающиеся вместе в запросах, размещаются в одном шарде, что снижает число межсерверных обращений в ходе обхода.

Третья известная стратегия — размещение по типу предиката, стандартная для триплетных хранилищ RDF — прямым измерениям в рамках настоящей работы не подвергалась. По теоретическим соображениям эта стратегия неустойчива к перекосу распределения типов рёбер: концентрация рёбер одного частотного типа на одном узле создаёт точку насыщения. Сопоставление с предикатным размещением вынесено в перспективы дальнейших испытаний.

4.6 Достоинства и недостатки разработанного решения

4.6.1 Достоинства

Разработанная система обладает следующими достоинствами:

- сочетание в одной системе графовой модели данных, горизонтального масштабирования, сериализуемой изоляции и внешнее согласованности одновременно, что не достигнуто ни одной из рассмотренных в Разделе 1 систем;
- быстрое обнаружение взаимоблокировок через гибридный детектор: быстрый путь обнаруживает циклы за время порядка единиц миллисекунд, что на один-два порядка быстрее централизованного поиска сильно связанных компонент;
- существенное снижение задержки обхода графа при семантическом размещении данных по сравнению с базовым хэш-размещением;
- отказоустойчивость с восстановлением обслуживания при отказе узлов: потеря одного голосующего узла хранилища метаданных

невидима клиенту, потеря узла плоскости данных приводит к восстановлению за время менее двух секунд;

- обязательное взаимное TLS-шифрование всех внутренних соединений, что обеспечивает защиту межузлового трафика по умолчанию.

4.6.2 Недостатки и ограничения

Разработанная система имеет следующие ограничения, выявленные в ходе тестирования:

- зависимость внешней согласованности от точности синхронизации времени: гарантия внешней согласованности справедлива только при условии, что реальное расхождение часов узлов не превышает заявленную границу неопределённости. При выходе из синхронизации точного времени гарантия может быть нарушена; контроль точности синхронизации возложен на операционные средства мониторинга, а не на саму систему;
- высокая задержка пагинации крупных результатов: каждая страница результата требует полного повторного исполнения обхода; при числе страниц порядка сотни общая задержка растёт линейно;
- ёмкость хранилища метаданных кластера ограничена пропускной способностью консенсус-группы из трёх голосующих узлов; при увеличении числа шардов выше порядка одной тысячи это становится узким местом для частоты обновлений карты шардов;

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы спроектировано и реализовано распределённое хранилище данных для семантической сети, обеспечивающее горизонтальное масштабирование, сериализуемую изоляцию транзакций, внешнюю согласованность операций фиксации и автоматическое семантическое распределение данных между узлами кластера.

В ходе выполнения работы получены следующие результаты.

- 1) Проведён сравнительный анализ распределённых систем управления данными, потенциально применимых для хранения графов знаний; рассмотрены три архитектурных класса систем, представленные восемью промышленными решениями. Установлено, что ни одна из рассмотренных систем не совмещает четыре ключевых свойства: графовую модель данных, горизонтальное распределение, сериализуемую изоляцию транзакций и внешнюю согласованность.
- 2) Составлен набор функциональных и нефункциональных требований, зафиксировавший целевую функциональность системы и целевые показатели качества.
- 3) Спроектирована архитектура системы, разделённая на три независимо масштабируемые плоскости: плоскость данных, управляющую плоскость и хранилище метаданных кластера. Обоснован выбор ключевых алгоритмов — журналируемого слиянного дерева для уровня хранения, диапазонного шардирования в сочетании с семантическим размещением сообществ по алгоритму Лейдена, протокола Raft для репликации, сериализуемой изоляции снимков, двухфазной фиксации по схеме Percolator, ожидания подтверждения над оракулом точного времени и гибридного детектора взаимоблокировок.

- 4) Реализован прототип системы. Уровень хранения и обработки данных реализован на языке Rust как единый исполняемый файл; управляющая плоскость реализована на языке Kotlin как два независимых сервиса без сохранения состояния — шлюз и оркестратор. Сетевые взаимодействия защищены обязательным взаимным TLS-шифрованием. Сквозное тестирование реализовано на языке Python.
- 5) Разработан комплекс автоматизированных тестов; покрытие — 78 % для модулей плоскости данных и 73 % для модулей управляющей плоскости, оба показателя превышают порог, установленный в задании.
- 6) Проведено нагрузочное тестирование и тестирование отказоустойчивости. Применение размещения по алгоритму Лейдена снижает задержку обхода графа на 46–52 % по сравнению с хэш-размещением. Восстановление лидерства консенсус-группы шарда укладывается в две секунды, восстановление ведущей реплики оркестратора — в пять секунд, потеря одного голосующего узла хранилища метаданных не приводит к перерыву обслуживания.
- 7) Выполнен сравнительный анализ предложенной стратегии размещения данных с базовыми подходами; задержка обхода при семантическом размещении приближается к нижней границе, задаваемой одношардной конфигурацией без межузловых обращений, при сопоставимой балансировке нагрузки между шардами.

Цель работы — обеспечение возможности горизонтального масштабирования хранилища данных семантической сети — достигнута. Возможность горизонтального масштабирования обеспечена архитектурно: каждый шард имеет собственную консенсус-группу; добавление и удаление узлов не требует изменений схемы данных и прикладного программного

интерфейса; перераспределение данных между узлами выполняется онлайн через механизм переходного консенсуса. Корректность сериализуемой изоляции и внешней согласованности подтверждена сквозными тестами с контролируемым созданием конфликтных сценариев.

Значимость работы состоит в систематизации подходов к построению распределённых графовых хранилищ с сильными транзакционными гарантиями и в трёх частных результатах. Первый — адаптация алгоритма Лейдена к режиму непрерывного перебалансирования распределённого графового хранилища: алгоритм выполняется итеративно над графом обращений клиентов, формируемым из текущих трассировок запросов, а решения о переносе вершин принимаются многокритериальной оптимизацией Парето с учётом стоимости миграции. Второй — гибридная архитектура обнаружения распределённых взаимоблокировок, объединяющая быстрый децентрализованный путь по схеме Чанди-Мисры-Хааса и медленный централизованный путь поиска сильно связанных компонент по Тарьяну, с детерминированной формулой выбора жертвы, согласующей решения обоих путей без распределённой координации. Третий — архитектурная композиция сериализуемой изоляции снимков, двухфазной фиксации Percolator, гибридных логических часов и оракула точного времени применительно к графовой распределённой системе с динамическим семантическим размещением данных. Полученные количественные показатели снижения задержки обходов при семантическом размещении применимы при выборе технологического стека для распределённых систем с графовыми моделями данных.

Среди направлений дальнейшего развития системы можно выделить следующие. Геораспределённое исполнение с использованием свидетельствующих узлов для обеспечения кворума в межрегиональных развёртываниях; механизм свидетельствующих узлов поддержан архитектурно, однако автоматическое управление размещением реплик с

учётom задержек между регионами составляет самостоятельную задачу. Совершенствование точности оракула точного времени через интеграцию с аппаратными источниками синхронизации и применение методов статистической оценки неопределённости. Расширение функциональности до поддержки фасада языков семейства W3C Semantic Web — стандартов RDF, SPARQL и OWL — поверх существующей модели размеченного графа свойств; разработка такого фасада возможна без изменения уровня хранения благодаря тому, что модель LPG является надмножеством триплетной модели. Расширение программы экспериментальных испытаний на дополнительные сценарии, включая набор тестов LDBC Social Network Benchmark и нагрузки, характерные для промышленных графов знаний.

Полученные результаты применимы при построении отказоустойчивых распределённых хранилищ графов знаний: горизонтальное масштабирование повышает ёмкость и пропускную способность хранилища, а семантическое размещение данных снижает накладные расходы на межузловые обращения при выполнении обходов графа.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Quillian M.R. Semantic memory // Semantic Information Processing / ed. M. Minsky. — Cambridge, MA: MIT Press, 1968. — P. 227–270.
2. Hogan A., Blomqvist E., Cochez M., d'Amato C., de Melo G., Gutierrez C., Kirrane S., Labra Gayo J.E., Navigli R., Neumaier S., Ngonga Ngomo A.-C., Polleres A., Rashid S.M., Rula A., Schmelzeisen L., Sequeda J., Staab S., Zimmermann A. Knowledge graphs // ACM Computing Surveys. — 2021. — Vol. 54, № 4. — Art. 71. — P. 1–37. — DOI: 10.1145/3447772.
3. Ji S., Pan S., Cambria E., Marttinen P., Yu P.S. A survey on knowledge graphs: representation, acquisition, and applications // IEEE Transactions on Neural Networks and Learning Systems. — 2022. — Vol. 33, № 2. — P. 494–514. — DOI: 10.1109/TNNLS.2021.3070843.
4. Волкова И.А., Шамаева Е.Д. Экскурс в графы знаний // International Journal of Open Information Technologies. — 2023. — Т. 11, № 5. — С. 75–82.
5. Wikidata Statistics [Электронный ресурс]: Wikimedia Foundation. — URL: <https://www.wikidata.org/wiki/Wikidata:Statistics> (дата обращения: 13.05.2026).
6. Чжан Цзяньхуа. Применение графов знаний как элемента интеллектуального производства (на примере обрабатывающей промышленности Китая) // Теория и практика общественного развития. — 2025. — № 2. — С. 133–139. — DOI: 10.24158/tipor.2025.2.17.
7. Chen H., Li C., Zheng C., Huang C., Fang J., Cheng J., Zhang J. G-Tran: a high performance distributed graph database with a decentralized architecture // Proceedings of the VLDB Endowment. — 2022. — Vol. 15, № 11. — P. 2545–2558. — DOI: 10.14778/3551793.3551813.
8. He T., Hu S., Lai L. [и др.]. GraphScope Flex: LEGO-like graph computing stack // Companion of the 2024 International Conference on

Management of Data (SIGMOD/PODS '24). — Santiago: ACM, 2024. — DOI: 10.1145/3626246.3653383.

9. Monteiro J., Sá F., Bernardino J. Experimental evaluation of graph databases: JanusGraph, Nebula Graph, Neo4j, and TigerGraph // Applied Sciences. — 2023. — Vol. 13, № 9. — Art. 5770. — DOI: 10.3390/app13095770.

10. Corbett J.C., Dean J., Epstein M. [и др.]. Spanner: Google's globally distributed database // Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI'12). — Hollywood: USENIX, 2012. — P. 251–264.

11. VanBenschoten N., Ajmani A., Gartner M., Matei A., Shah A., Sharif I., Shraer A., Storm A., Taft R., Tan O., Woods A., Walters P. Enabling the next generation of multi-region applications with CockroachDB // Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22). — Philadelphia: ACM, 2022. — P. 2701–2715. — DOI: 10.1145/3514221.3526053.

12. Yang Z., Yang C., Han F., Zhuang M., Yang B., Yang Z., Cheng X., Zhao Y., Shi W., Xi H., Yu H., Liu B., Pang Y., Song X., Wang Y., Wang Z., Lu Y., Tang X. [и др.]. OceanBase: a 707 million tpmC distributed relational database system // Proceedings of the VLDB Endowment. — 2022. — Vol. 15, № 12. — P. 3385–3397. — DOI: 10.14778/3554821.3554830.

13. RocksDB: A persistent key-value store for flash and RAM storage [Электронный ресурс]: официальная документация проекта. — URL: <https://rocksdb.org/> (дата обращения: 13.05.2026).

14. Неганов А.М. LSM-индекс с общими компонентами для индексации хронологических данных // Современные информационные технологии и ИТ-образование. — 2022. — Т. 18, № 2. — С. 337–352. — DOI: 10.25559/SITITO.18.202202.337-352.

15. Неганов А.М. Horizontally scalable LSM-tree based index for full-text search Boolean queries // Современные информационные технологии и

ИТ-образование. — 2022. — Т. 18, № 1. — С. 134–143. — DOI: 10.25559/SITITO.18.202201.134-143.

16. Gollapudi S., Karia N., Sivashankar V., Krishnaswamy R., Begwani N., Raz S., Lin Y., Zhang Y., Mahapatro N., Srinivasan P., Singh A., Simhadri H.V. Filtered-DiskANN: graph algorithms for approximate nearest neighbor search with filters // Proceedings of the ACM Web Conference 2023 (WWW '23). — Austin: ACM, 2023. — P. 3406–3416. — DOI: 10.1145/3543507.3583552.

17. Traag V.A., Waltman L., van Eck N.J. From Louvain to Leiden: guaranteeing well-connected communities // Scientific Reports. — 2019. — Vol. 9. — Art. 5233. — DOI: 10.1038/s41598-019-41695-z.

18. Sahu S. GVE-Leiden: fast Leiden algorithm for community detection in shared memory setting // Proceedings of the 53rd International Conference on Parallel Processing (ICPP '24). — Gotland: ACM, 2024. — P. 11–20. — DOI: 10.1145/3673038.3673146.

19. Deb K., Pratap A., Agarwal S., Meyarivan T. A fast and elitist multiobjective genetic algorithm: NSGA-II // IEEE Transactions on Evolutionary Computation. — 2002. — Vol. 6, № 2. — P. 182–197. — DOI: 10.1109/4235.996017.

20. Вихтенко Э.М., Зубков Д.В. Исследование алгоритмов NSGA-III и AGE-МОЕА-II для решения задач многокритериальной оптимизации // Инженерный вестник Дона. — 2024. — № 6.

21. Колесников А.А., Сербиновский М.Ю., Доценко В.Е. Параметрическая оптимизация конструкции на базе генетического алгоритма NSGA-II // Вестник Ростовского государственного университета путей сообщения. — 2022. — № 2. — С. 22–28. — DOI: 10.46973/0201-727X_2022_2_22.

22. Ongaro D., Ousterhout J. In search of an understandable consensus algorithm // Proceedings of the USENIX Annual Technical Conference (USENIX ATC'14). — Philadelphia: USENIX, 2014. — P. 305–319.
23. Du Y., Wang X. [и др.]. Multi-strategy-based leader election mechanism for the Raft algorithm // Concurrency and Computation: Practice and Experience. — 2023. — Vol. 35, № 22. — Art. e7734. — DOI: 10.1002/cpe.7734.
24. Булгаков В.Д., Гвоздевский И.Н. Модель и алгоритм консенсуса Proof of Performance // Программные системы и вычислительные методы. — 2024. — № 4. — DOI: 10.7256/2454-0714.2024.4.71119.
25. raft-rs: a Raft consensus algorithm implementation in Rust [Электронный ресурс]: репозиторий проекта TiKV. — URL: <https://github.com/tikv/raft-rs> (дата обращения: 13.05.2026).
26. Kulkarni S.S., Demirbas M., Madappa D., Avva B., Leone M. Logical physical clocks // Principles of Distributed Systems (OPODIS 2014). Lecture Notes in Computer Science. Vol. 8878. — Cham: Springer, 2014. — P. 17–32. — DOI: 10.1007/978-3-319-14472-6_2.
27. Kulkarni S.S., Appleton G., Nguyen D. Achieving causality with physical clocks // Proceedings of the 23rd International Conference on Distributed Computing and Networking (ICDCN '22). — Delhi: ACM, 2022. — P. 56–65. — DOI: 10.1145/3491003.3491009.
28. Cahill M.J., Fekete A., Röhm U. Serializable isolation for snapshot databases // Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data. — Vancouver: ACM, 2008. — P. 729–738. — DOI: 10.1145/1376616.1376690.
29. Alhomssi A., Leis V. Scalable and robust snapshot isolation for high-performance storage engines // Proceedings of the VLDB Endowment. — 2023. — Vol. 16, № 6. — P. 1426–1438. — DOI: 10.14778/3583140.3583157.

30. Peng D., Dabek F. Large-scale incremental processing using distributed transactions and notifications // Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI'10). — Vancouver: USENIX, 2010. — P. 251–264.

31. Huang D., Liu Q., Cui Q., Fang Z., Ma X., Xu F., Shen L., Tang L., Zhou Y., Huang M., Wei W., Liu C., Zhang J., Li J., Wu X., Song L., Sun R., Yu S., Zhao L., Cameron N., Pei L., Tang X. TiDB: a Raft-based HTAP database // Proceedings of the VLDB Endowment. — 2020. — Vol. 13, № 12. — P. 3072–3084. — DOI: 10.14778/3415478.3415535.

32. Chandy K.M., Misra J., Haas L.M. Distributed deadlock detection // ACM Transactions on Computer Systems. — 1983. — Vol. 1, № 2. — P. 144–156. — DOI: 10.1145/357360.357365.

33. Tarjan R.E. Depth-first search and linear graph algorithms // SIAM Journal on Computing. — 1972. — Vol. 1, № 2. — P. 146–160. — DOI: 10.1137/0201010.

34. Зимина К.И., Лапони́на О.Р. Механизмы межсервисной аутентификации в приложениях с микросервисной архитектурой // International Journal of Open Information Technologies. — 2023. — Т. 11, № 4. — С. 73–80.

35. Нестеренко А.Ю., Семенов А.М. Методика оценки безопасности криптографических протоколов // Прикладная дискретная математика. — 2022. — № 56. — С. 33–49. — DOI: 10.17223/20710410/56/4.

ПРИЛОЖЕНИЕ А

Фрагменты определений программных интерфейсов

Полное определение программных интерфейсов системы доступно в репозитории, ссылка на который приведена в Приложении Б.

Листинг А.1 – Фрагмент определения интерфейса транзакций в формате protobuf

```
syntax = "proto3";
package rhizome.v1;

import "rhizome/v1/common.proto";

service TransactionService {
  rpc BeginTransaction (BeginTxRequest)
    returns (BeginTxResponse);

  rpc CommitTransaction (CommitTxRequest)
    returns (CommitTxResponse);

  rpc AbortTransaction (AbortTxRequest)
    returns (AbortTxResponse);
}

enum IsolationLevel {
  ISOLATION_LEVEL_UNSPECIFIED = 0;
  ISOLATION_LEVEL_READ_COMMITTED = 1;
  ISOLATION_LEVEL_SNAPSHOT = 2;
  ISOLATION_LEVEL_SERIALIZABLE = 3;
}

message BeginTxRequest {
  IsolationLevel isolation = 1;
  optional bytes idempotency_key = 2;
}

message BeginTxResponse {
  TxHandle handle = 1;
  Hlc start_ts = 2;
}

message TxHandle {
  bytes tx_id = 1;
  string node_id = 2;
}
```

```

message CommitTxRequest { TxHandle handle = 1; }

message CommitTxResponse {
  Hlc      commit_ts = 1;
  CommitInfo info     = 2;
}

message CommitInfo {
  bool    serializable_ok      = 1;
  uint32 participating_shards  = 2;
  bool    external_consistency_ok = 3;
}

message AbortTxRequest { TxHandle handle = 1; }
message AbortTxResponse { string reason = 1; }

```

Листинг А.2 – Фрагмент схемы GraphQL шлюза, описывающий операции жизненного цикла транзакции и операции чтения и записи с указанием транзакционного контекста

```

enum IsolationLevel {
  READ_COMMITTED
  SNAPSHOT
  SERIALIZABLE
}

input TxHandleInput {
  txId    : String!
  nodeId  : String!
}

type Mutation {
  beginTransaction(
    isolation : IsolationLevel = SERIALIZABLE
  ): TxHandle!

  commitTransaction(tx: TxHandleInput!): CommitResult!

  abortTransaction(tx: TxHandleInput!): AbortResult!

  createVertex(
    input : VertexInput!
    tx    : TxHandleInput
  ): Vertex!

  createEdge(

```

```

        input : EdgeInput!
        tx     : TxHandleInput
    ): Edge!
}

type Query {
    vertex(id: ID!, tx: TxHandleInput): Vertex

    traverse(
        from      : ID!
        depth     : Int!
        filter     : EdgeFilter
        tx        : TxHandleInput
    ): TraversalResult!

    knn(
        query      : VectorInput!
        k          : Int!
        filter     : VertexFilter
        tx         : TxHandleInput
    ): [Vertex!]!
}

type CommitResult {
    commitTs           : String!
    serializableOk     : Boolean!
    externalConsistency : Boolean!
    participatingShards : Int!
}

```


ПРИЛОЖЕНИЕ Б

Разработанное программное обеспечение

Полный код прототипа и сопутствующая документация размещены в открытом репозитории по адресу <https://github.com/maxbarsukov/rhizome>.

Репозиторий содержит реализацию распределённого графового хранилища данных с поддержкой сериализуемой изоляции транзакций и внешней согласованности. В частности, включены:

- исходные файлы модулей плоскости данных на языке Rust: уровень хранения на основе RocksDB, уровень репликации, менеджер транзакций с реализацией сериализуемой изоляции снимков, двухфазной фиксации Percolator и ожидания подтверждения, подсистема графовых вычислений и менеджер схемы данных;
- исходные файлы модулей управляющей плоскости на языке Kotlin: шлюз с интерфейсом GraphQL и стоимостной маршрутизацией запросов, оркестратор с циклами размещения шардов по алгоритму Лейдена и медленным путём обнаружения распределённых взаимоблокировок;
- protobuf-определения межкомпонентных gRPC-интерфейсов;
- конфигурация Docker Compose для эталонного развёртывания кластера;
- набор автоматических тестов на языках Rust, Kotlin и Python (модульные, интеграционные и сквозные сценарии);
- документация архитектуры, эксплуатации и безопасности системы, журнал изменений.